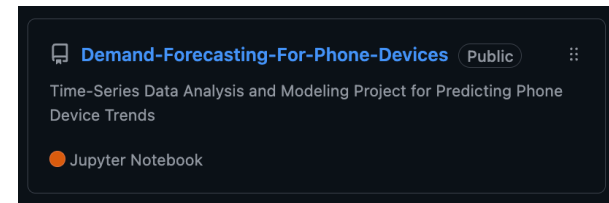
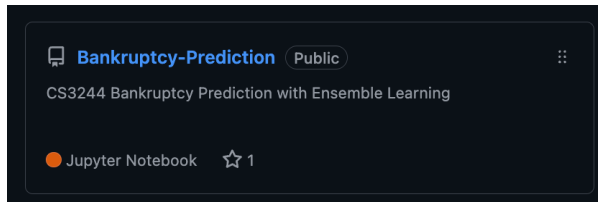


MLOps for Traditional Machine Learning

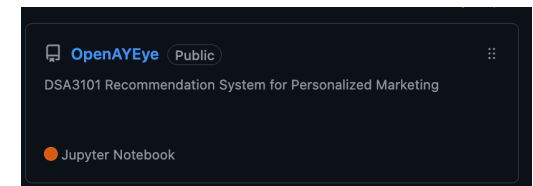
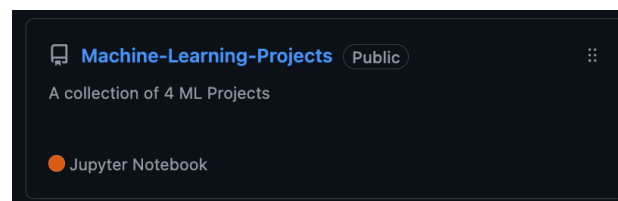
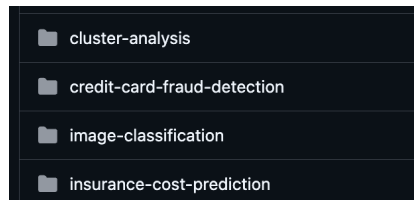
DSA4288 Honours Project in Data Science and Analytics

By: Chin Sek Yi
Supervisor: Prof. Markus Kirchberg

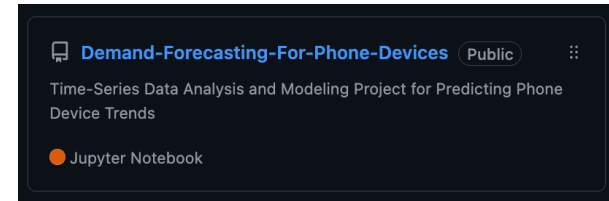
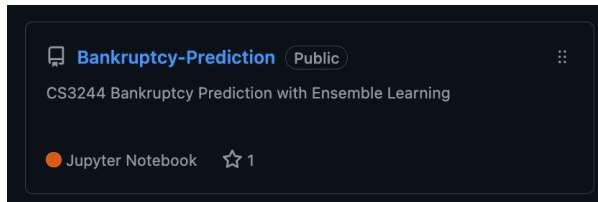
How many of you have built an amazing ML model
that never saw production?



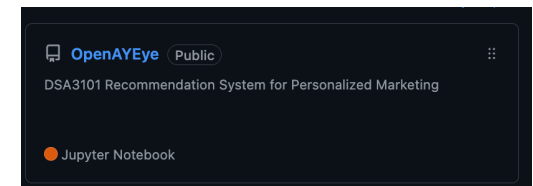
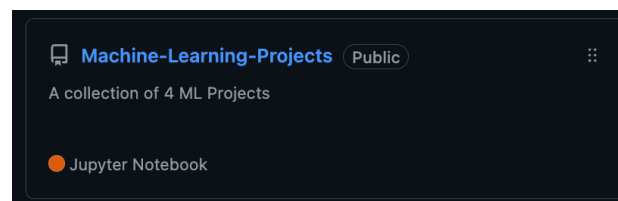
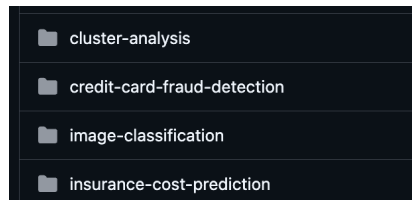
How many of you have built an amazing ML model
that never saw production?



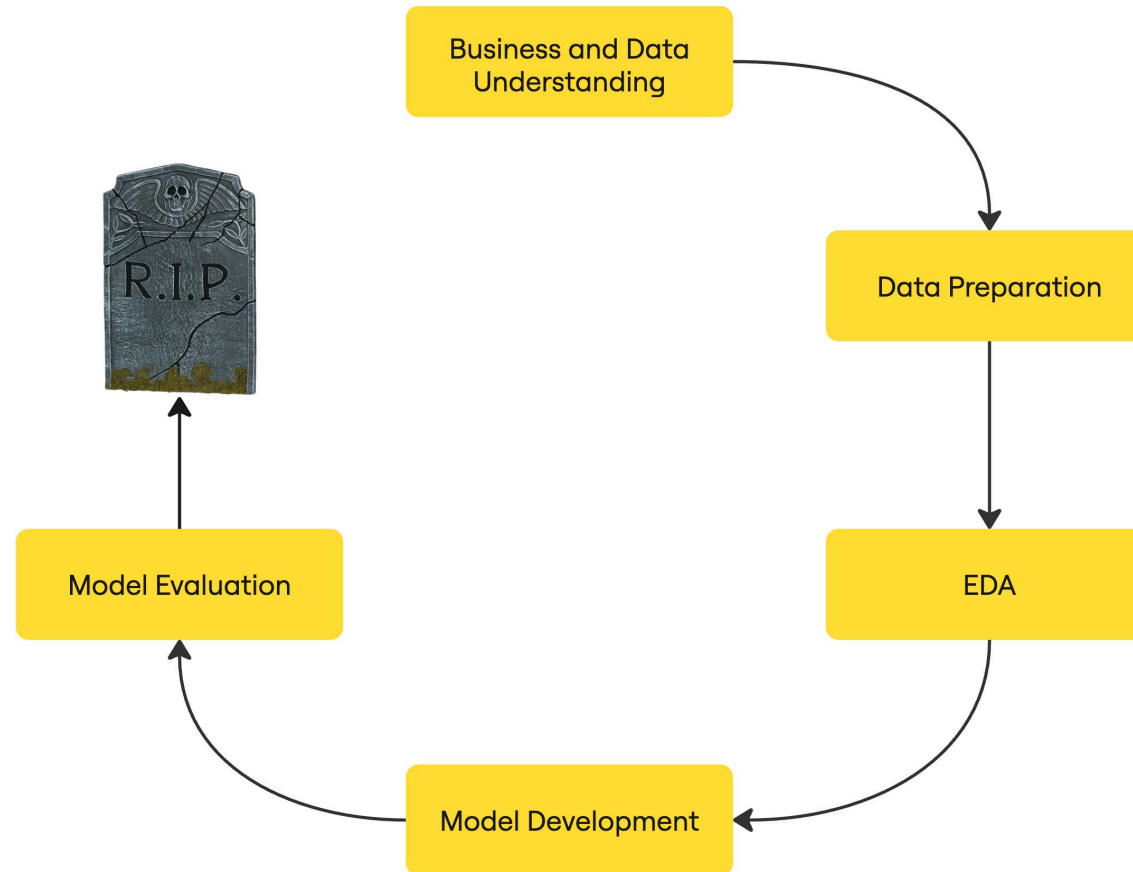
Problem Statement



7+ ML projects, 0 in production !



Problem Statement



Objective

Machine Learning Operations (MLOps)

Roadmap

1. MLOps Motivation

2. System Overview & Use Case — Building the Foundation

3. Pipeline Component — The Data & Model Pipeline

4. Infrastructure & Deployment— Scaling Across Environments

Demo

Roadmap

1. MLOps Motivation



2. System Overview & Use Case — Building the Foundation

3. Pipeline Component — The Data & Model Pipeline

4. Infrastructure & Deployment— Scaling Across Environments

Demo

1.1 What is MLOps? (Machine Learning Operations)

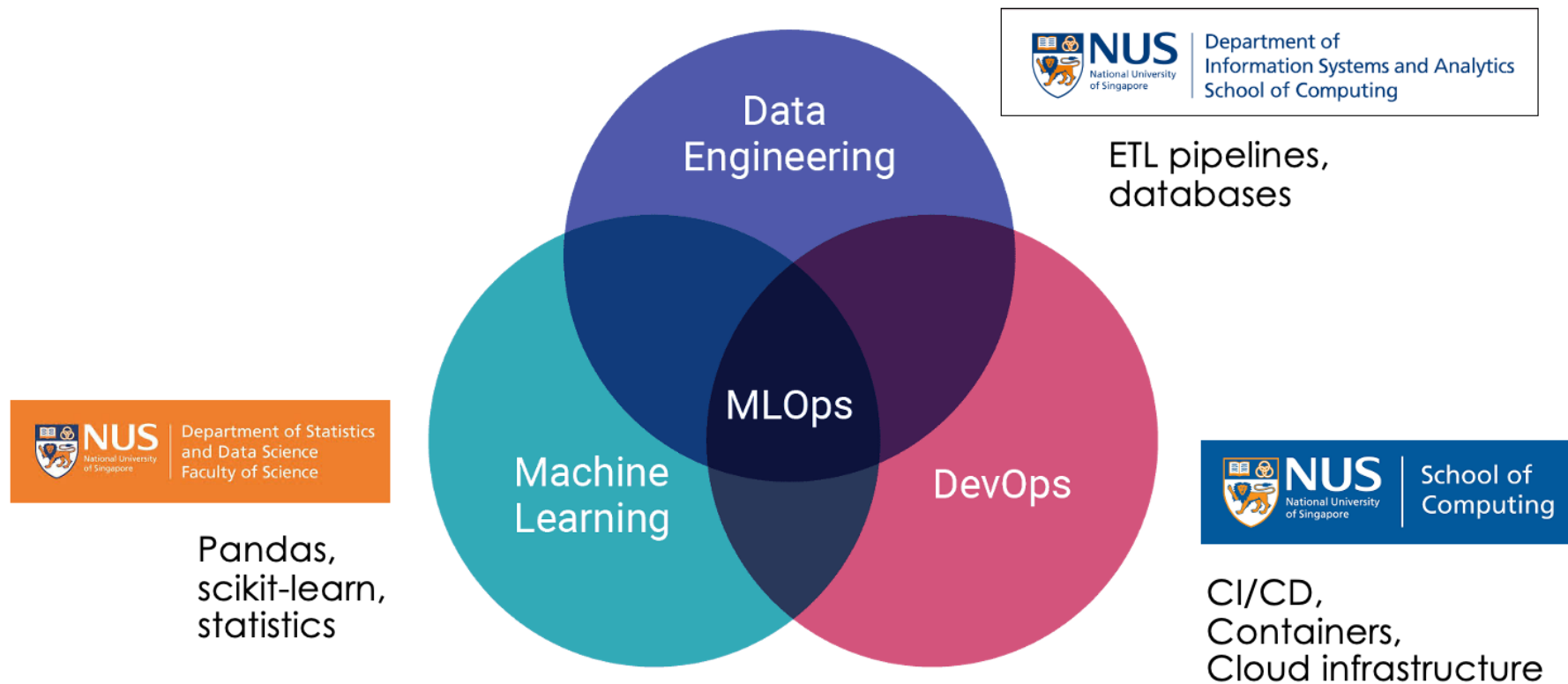
Automates and standardizes the entire ML lifecycle—from development to deployment and monitoring.

Key Features:

- End-to-end automation
- CI/CD for ML models
- Testing & monitoring
- DevOps for ML
- Reliable, production-ready systems

ML models are only as good as the systems that support them

1.2 The Skills Gap: Why Universities Don't Teach MLOps



1.3 MLOps: Bridging Two Worlds

Traditional Data Science Workflow

- Manual experimental tracking
- Jupyter notebook development
- Offline training and evaluation
- Individual researcher workflow

DevOps Workflow

- Development → Staging → Production
- CI/CD automation
- Code reviews & Pull requests
- Containerisation and Cloud infrastructure



MLOps = Both Combined

Experimental rigor + Production reliability = Production-ready ML systems

Roadmap

1. MLOps Motivation



2. System Overview & Use Case — Building the Foundation

3. Pipeline Component — The Data & Model Pipeline

4. Infrastructure & Deployment— Scaling Across Environments

Demo

Roadmap

1. MLOps Motivation

2. System Overview & Use Case — Building the Foundation

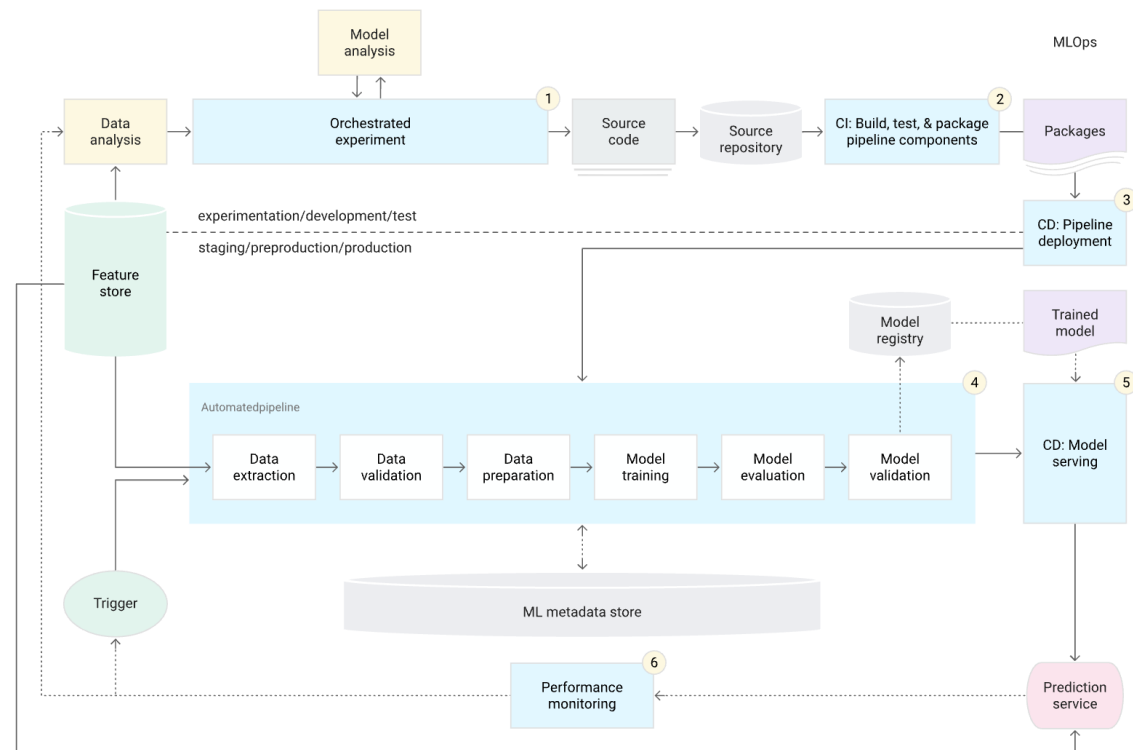


3. Pipeline Component — The Data & Model Pipeline

4. Infrastructure & Deployment— Scaling Across Environments

Demo

2.1 Gold Standard: MLOps Architecture diagram (Google Cloud)



Source: Google cloud

Business use case?

2.2 Why Fraud Detection?

The Ultimate MLOps Stress Test

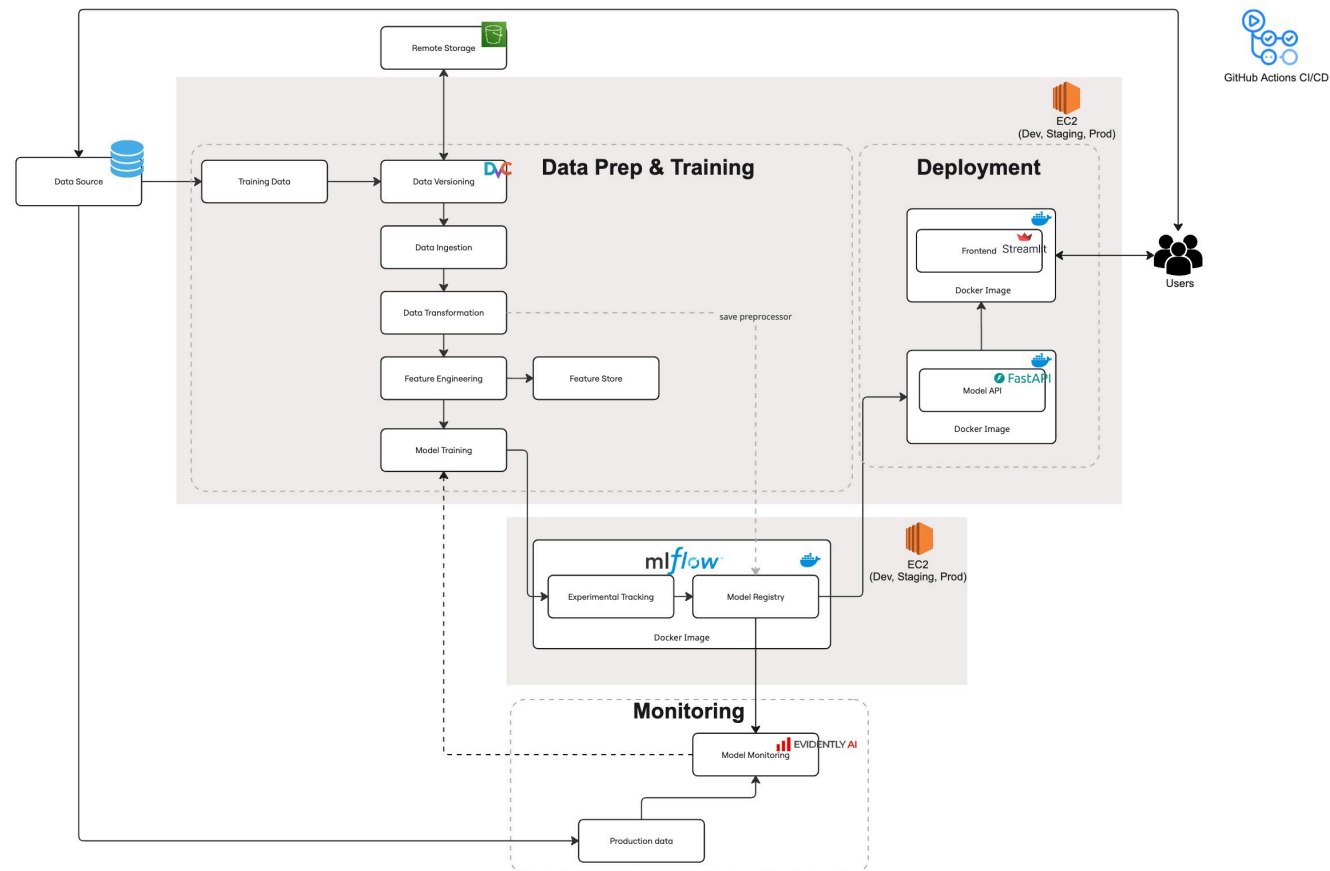
- Rapidly changing fraud patterns
- High stakes: real money, real consequences
- Need for fast, reliable model updates



step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
544	CASH_OUT	412668.39	C84071102	412668.39	0.0	C1576697216	1445047.47	1857715.86	1	0

Synthetic Fraud Dataset (Paysim)

2.3 My MLOps System Architecture



Roadmap

1. MLOps Motivation

2. System Overview & Use Case — Building the Foundation



3. Pipeline Component — The Data & Model Pipeline

4. Infrastructure & Deployment— Scaling Across Environments

Demo

Roadmap

1. MLOps Motivation

2. System Overview & Use Case — Building the Foundation

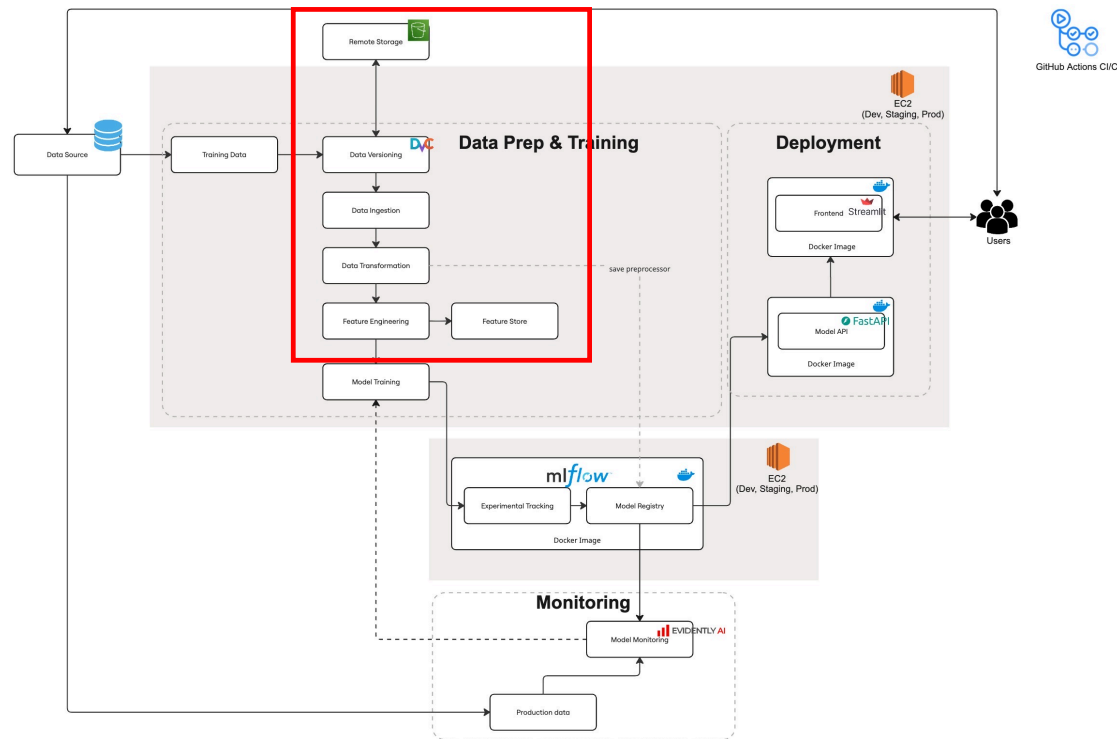
3. Pipeline Component — The Data & Model Pipeline



4. Infrastructure & Deployment— Scaling Across Environments

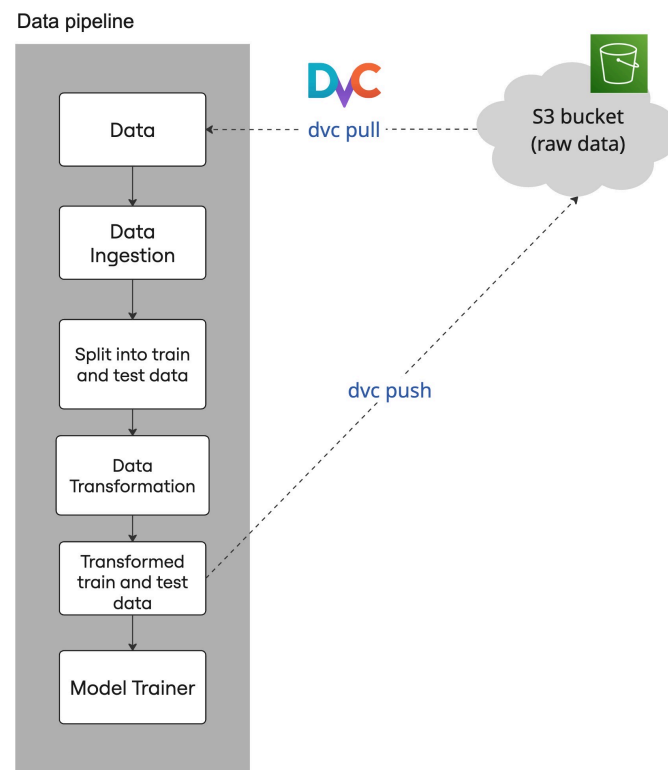
Demo

3.1 Data Preparation & Training



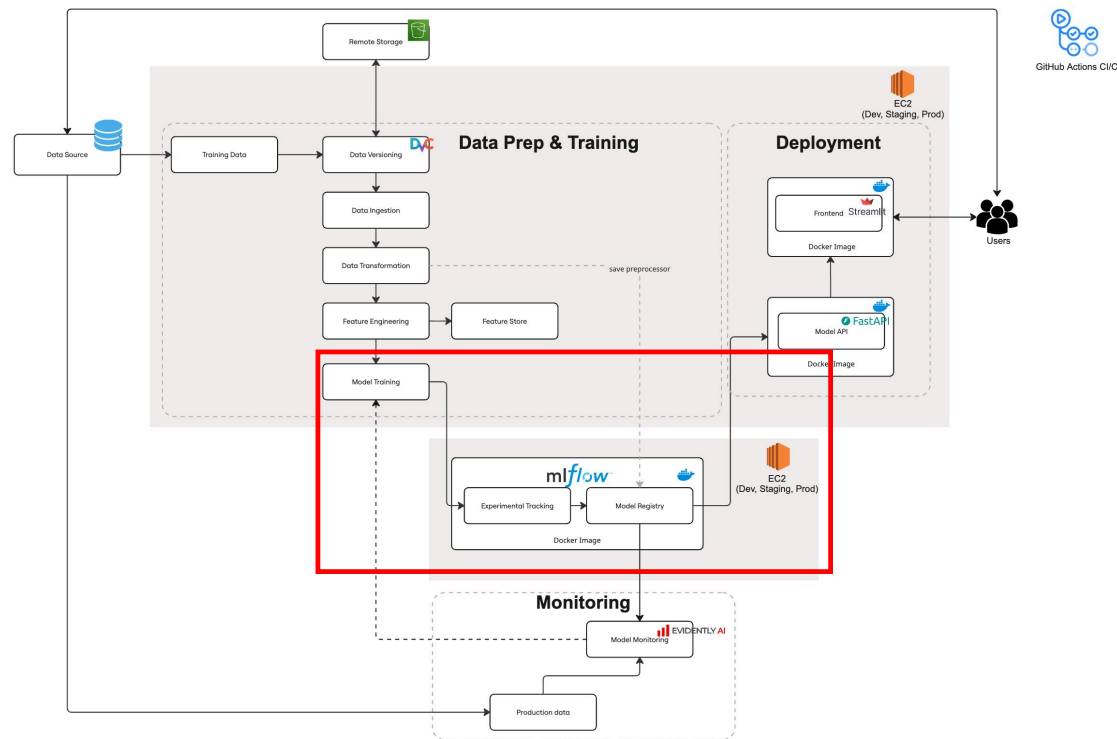
- Data management with DVC
- Data pipeline
- Model training and experimental tracking
- Model registry

3.1.1 Data Management & Versioning



1. Pull and push data to remote storage
2. Data versioning

3.1 Data Preparation & Training



- Data management with DVC
- Data pipeline
- Model training and experimental tracking
- Model registry

3.1.2 Model Training: Experimental Tracking

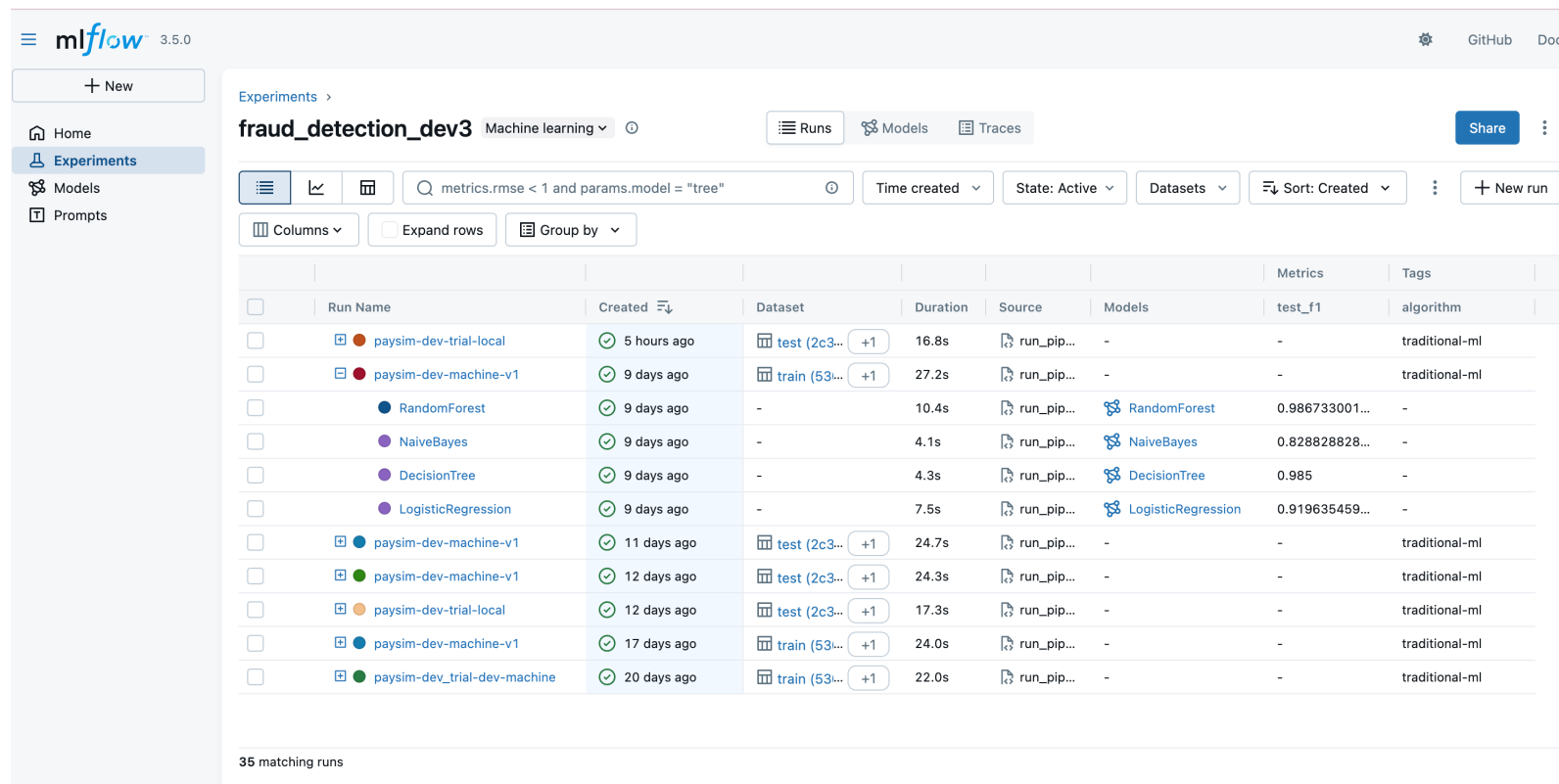
Problem: Manual, error-prone, not reproducible

RESULTS					metrics	
Run_id	Model	Standardisation	Datatype	Fe	Calinski-Harabasz (max)	Notes
baseline	77a25b4e89f K-Means	Standard Scalar	numerical		1457	cluster_n = 7 is the best for silhouette, david, calinski, but not for inertia
best	bb4a55be19 K-Prototype	Standard Scalar	mixed		1141	
	0a41c5f612b K-Prototype, optuna (composite)	Standard Scalar	mixed		1227.73	from Optuna_Kprototype_Study_for_Composite_Metric. Composite score = 1227.73
	4bdebd639d K-Prototype, optuna (best silhouette)	Standard Scalar	mixed		1130.8	from Optuna_Kprototype_Study_for_each_metrics
	d8734d39e3f K-Prototype, optuna (best Davies-Bouldin)	Standard Scalar	mixed		1010.4371	
	7245c4909c K-Prototype, optuna (best Calinski-Harabasz)	Standard Scalar	mixed		1212.5887	
	DBSCAN	Standard Scalar	numerical		1.493	NA because device type is separated
	AgglomerativeClustering	Standard Scalar	mixed		2386.103	
	KMedoid	Standard Scalar	mixed		2153.153	
					1447.742	
					1163.834	
OUTDATED!					CAVEATS	
BEST MODEL		EXPLANATION				
1	K-Prototype	Good Silhouette Score (0.555) -> In Lowest Davies-Bouldin Score (0.52) Highest Calinski-Harabasz Score (7 Works on Mixed Data				cluster variance -> well-separated clusters.
2	AgglomerativeClustering (cosine)	Highest silhouette score (0.763) -> s Very good Calinski-Harabasz (2386.1) -> clusters are well-separated. Davies-Bouldin isn't the lowest, but still acceptable (0.974). Clearly defined optimal_k = 2, , and cosine distance seems best for this structure. Works on Mixed Data				k=2 doesn't make sense (take higher k, sacrificing some accuracy)



3.1.2 Model Training: Experimental Tracking

Solution: MLflow



The screenshot displays the MLflow Experiments page for the 'fraud_detection_dev3' experiment. The interface includes a sidebar with navigation links (Home, Experiments, Models, Prompts) and a top navigation bar with 'mlflow 3.5.0', 'GitHub', and 'Docs' links. The main content area shows a table of runs with columns for Run Name, Created, Dataset, Duration, Source, Models, Metrics, and Tags. The table lists 12 runs, including 'paysim-dev-trial-local', 'paysim-dev-machine-v1', and various ML models like RandomForest, NaiveBayes, DecisionTree, and LogisticRegression. Each run entry includes a checkbox, a colored circle representing the run's status, and a green checkmark indicating successful completion. The 'Created' column shows the time elapsed since the run was created (e.g., '5 hours ago', '9 days ago'). The 'Dataset' column shows the dataset used for training (e.g., 'test (2c3...)', 'train (53...)', '-'). The 'Duration' column shows the time taken to complete the run (e.g., '16.8s', '27.2s'). The 'Source' column shows the source of the run (e.g., 'run_pip...'). The 'Models' column shows the models trained during the run (e.g., 'RandomForest', 'NaiveBayes', 'DecisionTree', 'LogisticRegression'). The 'Metrics' column shows the 'test_f1' score (e.g., '0.986733001...', '0.828828828...'). The 'Tags' column shows the 'algorithm' tag (e.g., 'traditional-ml').

	Run Name	Created	Dataset	Duration	Source	Models	Metrics	Tags
☐	paysim-dev-trial-local	5 hours ago	test (2c3...)	16.8s	run_pip...	-	-	traditional-ml
☐	paysim-dev-machine-v1	9 days ago	train (53...)	27.2s	run_pip...	-	-	traditional-ml
☐	RandomForest	9 days ago	-	10.4s	run_pip...	RandomForest	0.986733001...	-
☐	NaiveBayes	9 days ago	-	4.1s	run_pip...	NaiveBayes	0.828828828...	-
☐	DecisionTree	9 days ago	-	4.3s	run_pip...	DecisionTree	0.985	-
☐	LogisticRegression	9 days ago	-	7.5s	run_pip...	LogisticRegression	0.919635459...	-
☐	paysim-dev-machine-v1	11 days ago	test (2c3...)	24.7s	run_pip...	-	-	traditional-ml
☐	paysim-dev-machine-v1	12 days ago	test (2c3...)	24.3s	run_pip...	-	-	traditional-ml
☐	paysim-dev-trial-local	12 days ago	test (2c3...)	17.3s	run_pip...	-	-	traditional-ml
☐	paysim-dev-machine-v1	17 days ago	train (53...)	24.0s	run_pip...	-	-	traditional-ml
☐	paysim-dev_trial-dev-machine	20 days ago	train (53...)	22.0s	run_pip...	-	-	traditional-ml

35 matching runs

Each experimental run trains 4 ML models. Logs metrics, preprocessor and model artifacts, etc.

3.1.2 Model Training: Experimental Tracking

Solution: MLflow

The screenshot displays the MLflow 3.5.0 web interface. The left sidebar contains navigation links: Home, Experiments (selected), Models, and Prompts. The main content area shows the details for a run named 'RandomForest' under the experiment 'fraud_detection_dev3'. The 'Overview' tab is active, showing a description (none), metrics (5), and parameters (20). The metrics table lists cv_accuracy, test_precision, test_f1, test_accuracy, and test_recall, all with values around 0.98-0.99. The parameters table lists model_type (RandomForest), bootstrap (True), and ccp_alpha (0.0). The right sidebar provides 'About this run' information, including creation time, user (ubuntu), experiment ID, status (Finished), run ID, duration, parent run, and source. It also shows 'Registered models' with 'dev.fraud-detection-model v36'.

Metrics (5)

Metric	Value	Models
cv_accuracy	0.9817857142857143	RandomForest
test_precision	0.9818481848184818	RandomForest
test_f1	0.9867330016583747	RandomForest
test_accuracy	0.9866666666666667	RandomForest
test_recall	0.9916666666666667	RandomForest

Parameters (20)

Parameter	Value
model_type	RandomForest
bootstrap	True
ccp_alpha	0.0

About this run

Created at: 10/28/2025, 05:43:57 PM
Created by: ubuntu
Experiment ID: 1
Status: Finished
Run ID: b976ab9dfe9d4baba7baf00238a17426
Duration: 10.4s
Parent run: paysim-dev-machine-v1
Source: run_pipeline.py
Registered prompts: —

Datasets
None

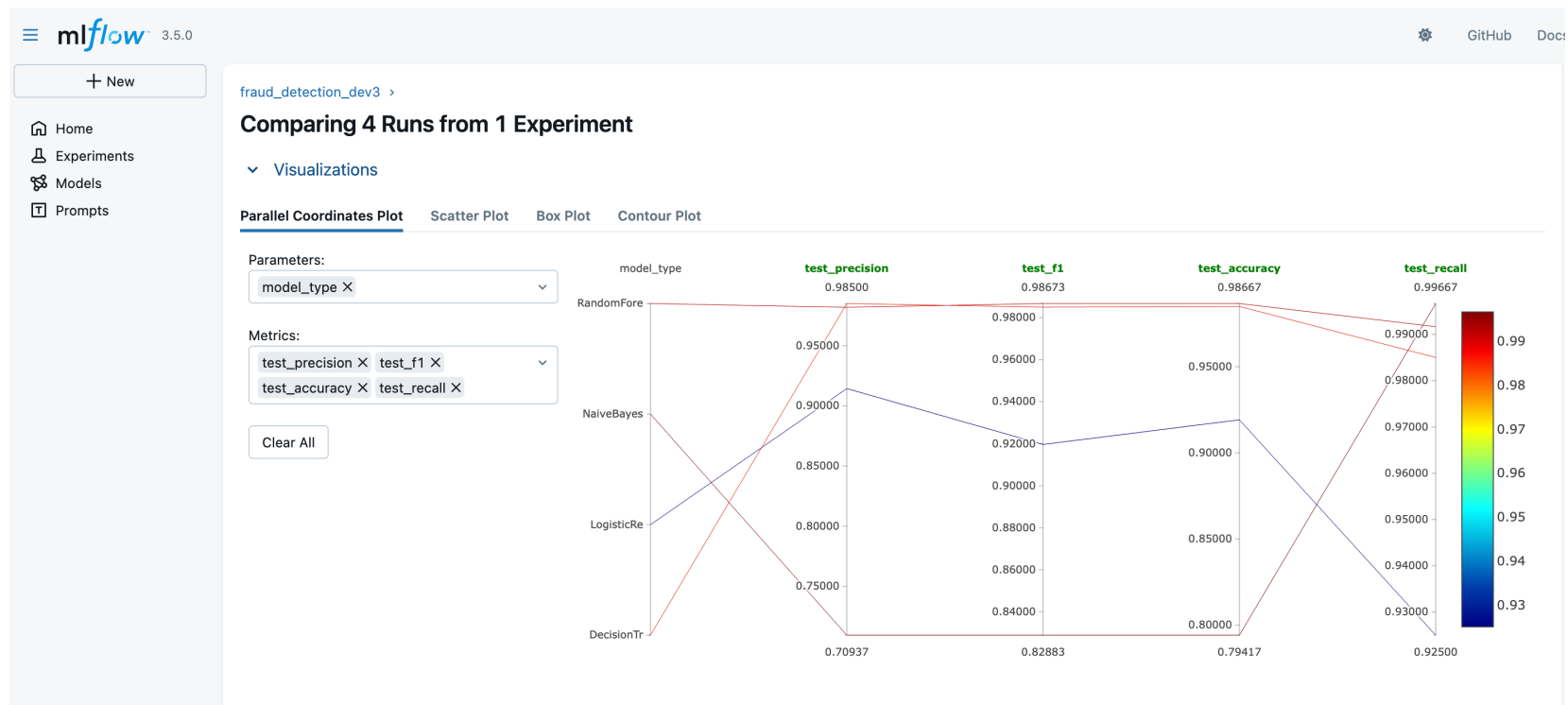
Tags
model_name: RandomForest

Registered models
dev.fraud-detection-model v36

View a specific model – see metrics and parameters

3.1.2 Model Training: Experimental Tracking

Solution: MLflow



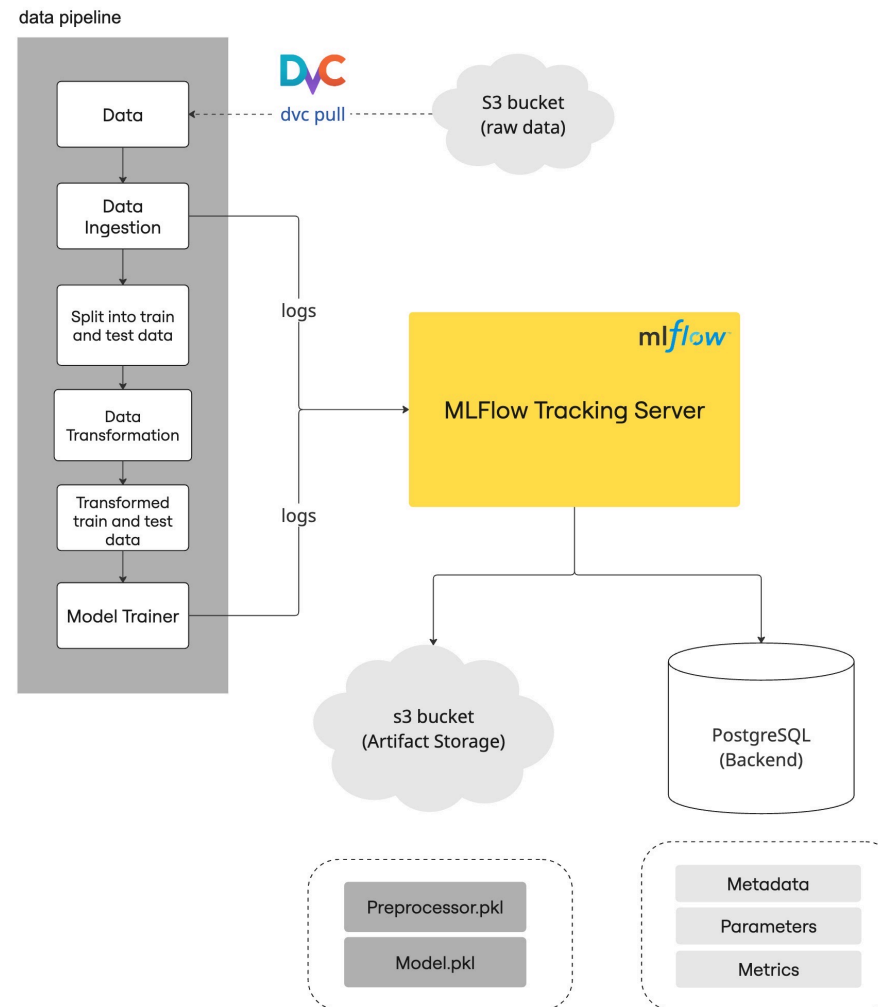
Easily compare metrics of trained models

3.1.3 Model Registry: Choose the best model

The screenshot shows the mlflow 3.5.0 Model Registry interface. The left sidebar contains navigation links: Home, Experiments, Models (selected), and Prompts. The main content area displays the 'dev.fraud-detection-model' with its creation and modification timestamps. Below this, there are sections for Description, Tags, and Versions. The Versions section includes a 'Compare' button and a table of model versions. The table has columns for Version, Registered at, Created by, Tags, Aliases, and Description. The 'Version 42' row is highlighted with a yellow box, and an arrow points to the '@ champion' alias. A trophy icon is also present.

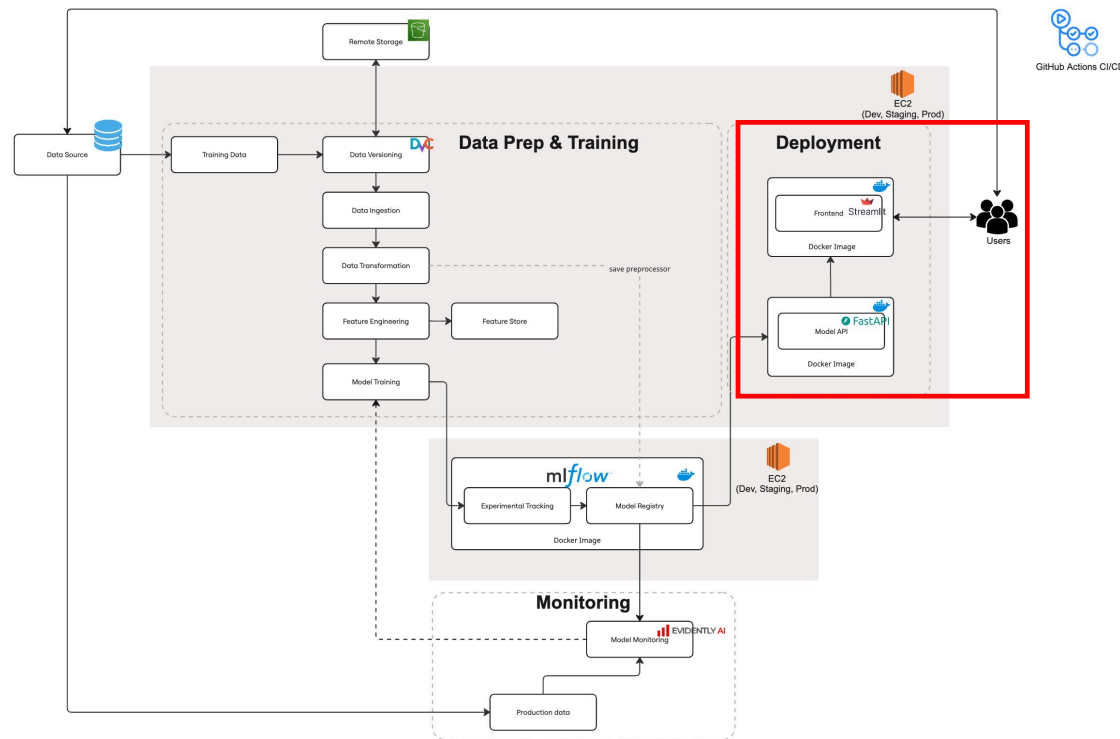
Version	Registered at	Created by	Tags	Aliases	Description
Version 44	11/07/2025, 10:42:32 AM		Add	Add	
Version 43	11/07/2025, 10:42:27 AM		Add	Add	
Version 42	11/07/2025, 10:42:24 AM		Add	@ champion	
Version 41	11/07/2025, 10:42:20 AM		Add	@ candidate	
Version 40	11/07/2025, 10:40:35 AM		Add	Add	
Version 39	11/07/2025, 10:40:27 AM		Add	Add	
Version 38	11/07/2025, 10:40:23 AM		Add	Add	
Version 37	11/07/2025, 10:40:19 AM		Add	Add	
Version 36	10/28/2025, 05:44:07 PM		Add	Add	

The best model is labelled as “champion”

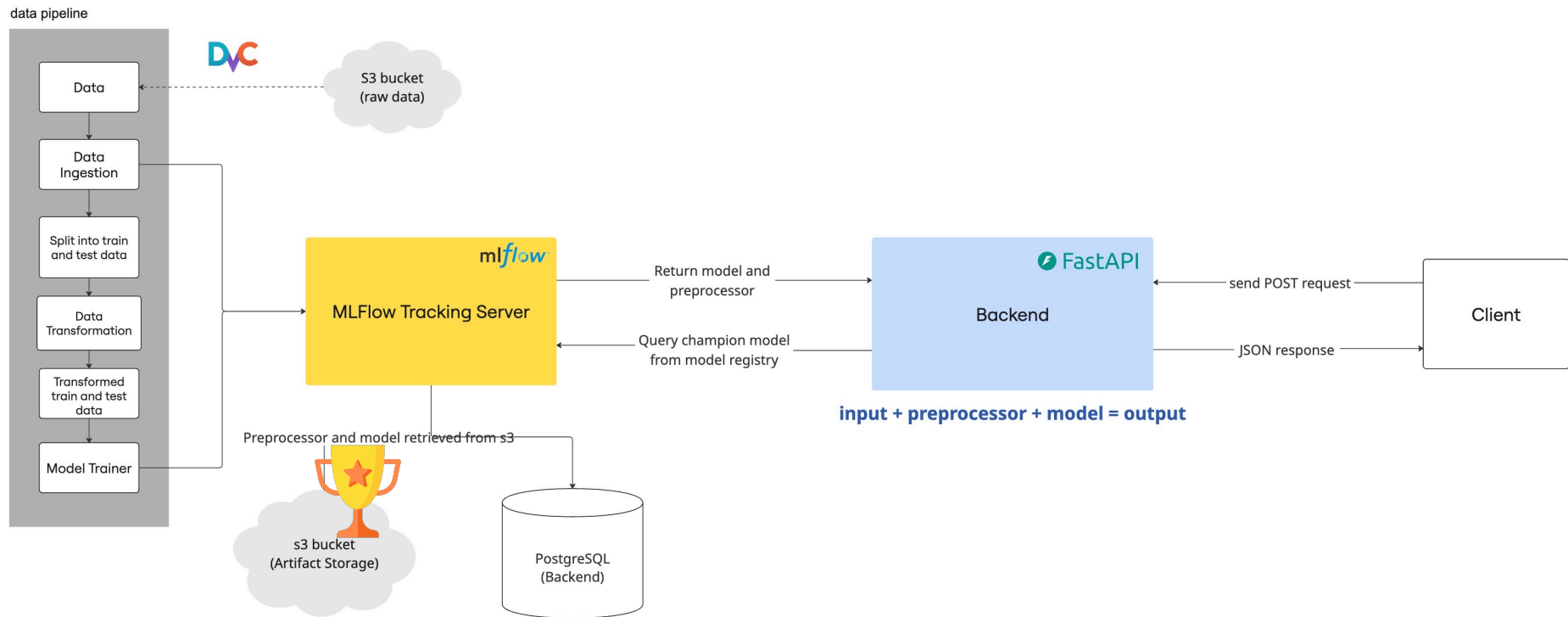


3.2 Deployment

- FastAPI backend
- Streamlit frontend



3.2.1 Backend: FastAPI



3.2.2 Frontend: Streamlit

Deploy ⋮

Select Page

☒ Predict

☐ Model Info

☐ Metrics

☐ Feature Info



Fraud Detection Demo

Interact with the fraud detection model served by FastAPI + MLflow.

Single Transaction Prediction

Load sample input?

☐ None ☒ Non-Fraud Example ☐ Fraud Example

step (time step - must be positive integer)	\$amount	\$oldbalanceOrig
1 - +	1000.00 - +	5000.00 - +
\$newbalanceOrig	\$oldbalanceDest	\$newbalanceDest
4000.00 - +	0.00 - +	1000.00 - +
Transaction type	nameOrig (format: C1234567890)	nameDest (format: C1234567890)
CASH_OUT ▾	C84071102	C1576697216

Predict

Prediction: 0 (1 = Fraud, 0 = Legit)

✓

Version 42

11/07/2025, 10:42:24 AM

Add

@ champion



Deploy

Select Page

- ☐ Predict
- ☒ Model Info
- ☐ Metrics
- ☐ Feature Info



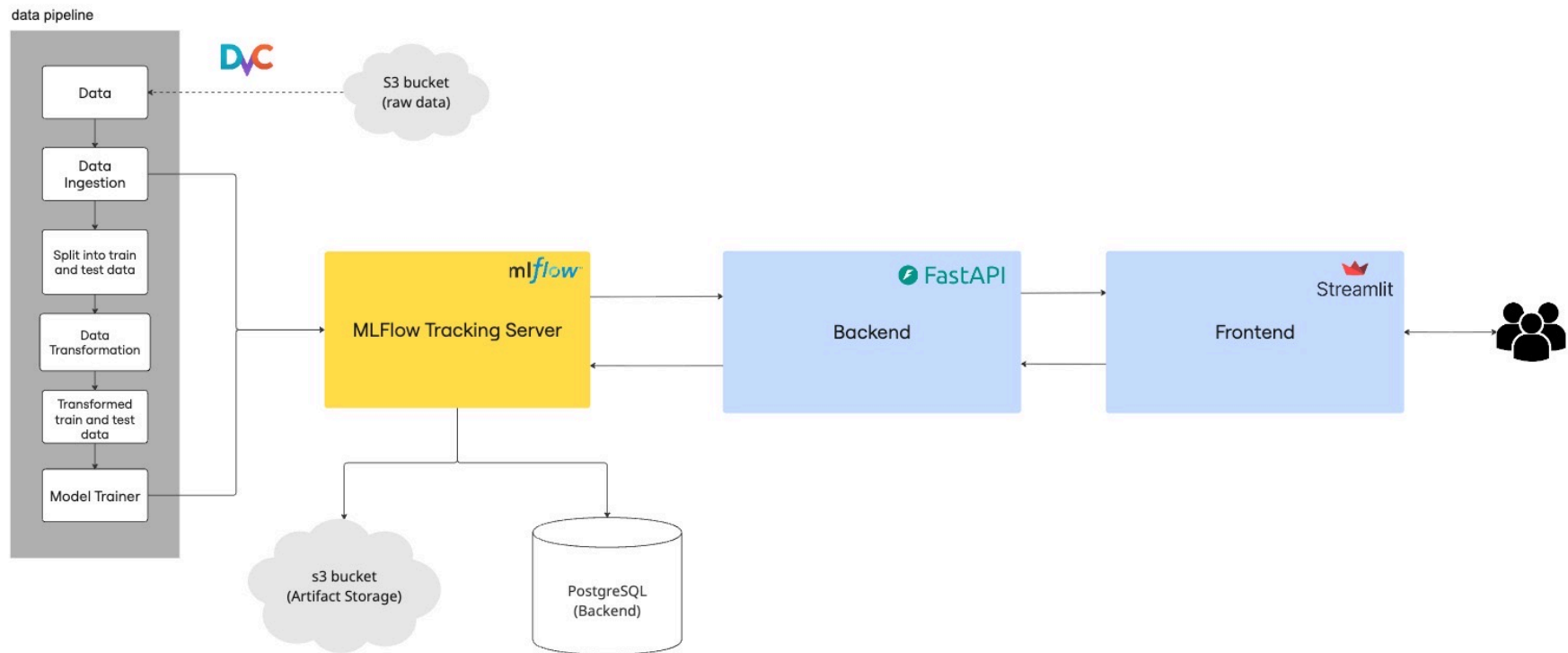
Fraud Detection Demo

Interact with the fraud detection model served by FastAPI + MLflow.

Model Metadata

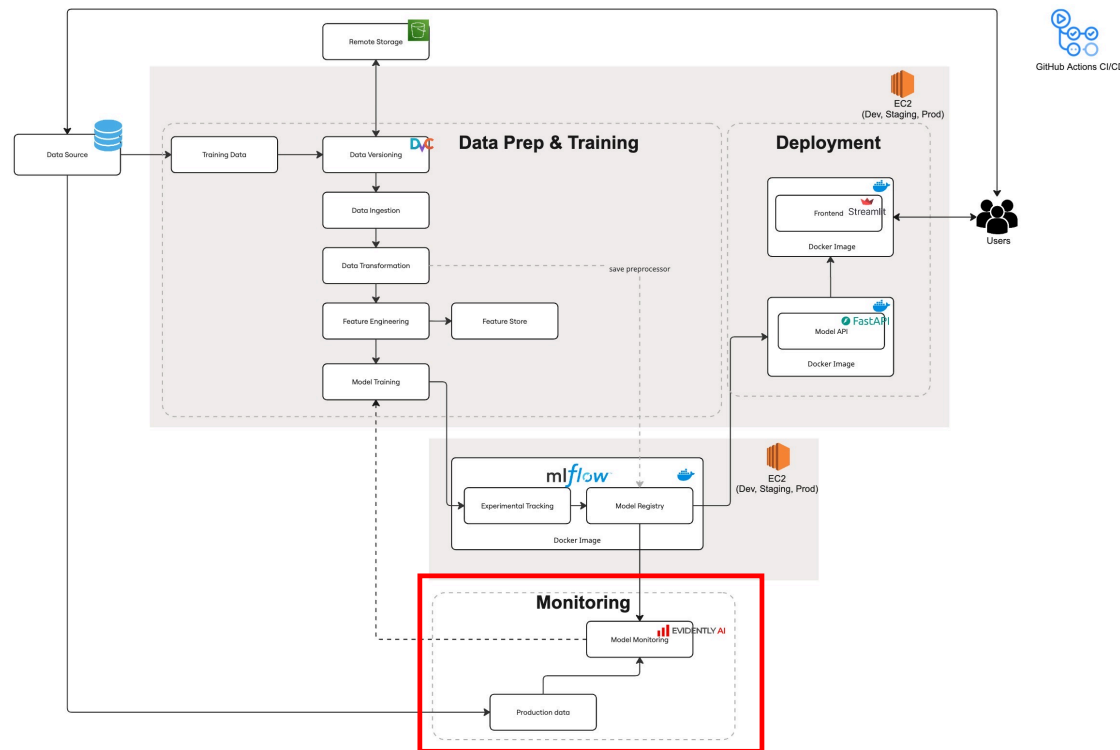
Fetch Model Info

```
{
  "model_name" : "dev.fraud-detection-model"
  "model_name_tag" : "DecisionTree"
  "version" : "42"
  "alias" : "champion"
  "run_id" : "2fe39da1d3f2415988af9f60b6bb50af"
  "status" : "READY"
  "creation_timestamp" : 1762483344095
  "mlflow_tracking_uri" : "http://10.0.1.25:5050"
  "model_trained_on" : "paysim-dev.csv"
  "environment" : "dev"
}
```



3.3 Monitoring

- Model monitoring and drift detection with Evidently AI



3.3 Model monitoring and drift detection

- Track model performance in production
- Detect data drift and concept drift
- Automated alerts for performance drops, and model retraining
- Use Evidently AI for monitoring and visualisation

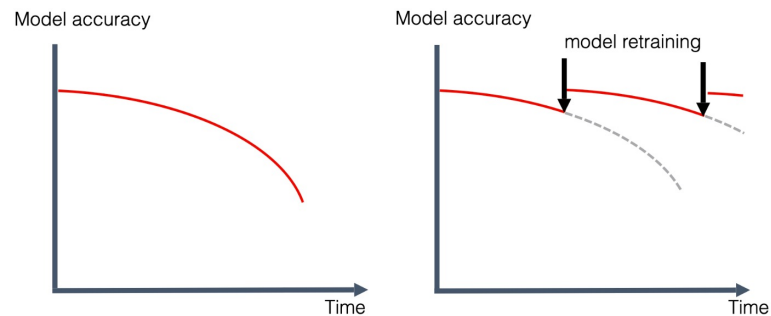


Image source: Evidently AI docs

Roadmap

1. MLOps Motivation

2. System Overview & Use Case — Building the Foundation

3. Pipeline Component — The Data & Model Pipeline



4. Infrastructure & Deployment— Scaling Across Environments

Demo

Roadmap

1. MLOps Motivation

2. System Overview & Use Case — Building the Foundation

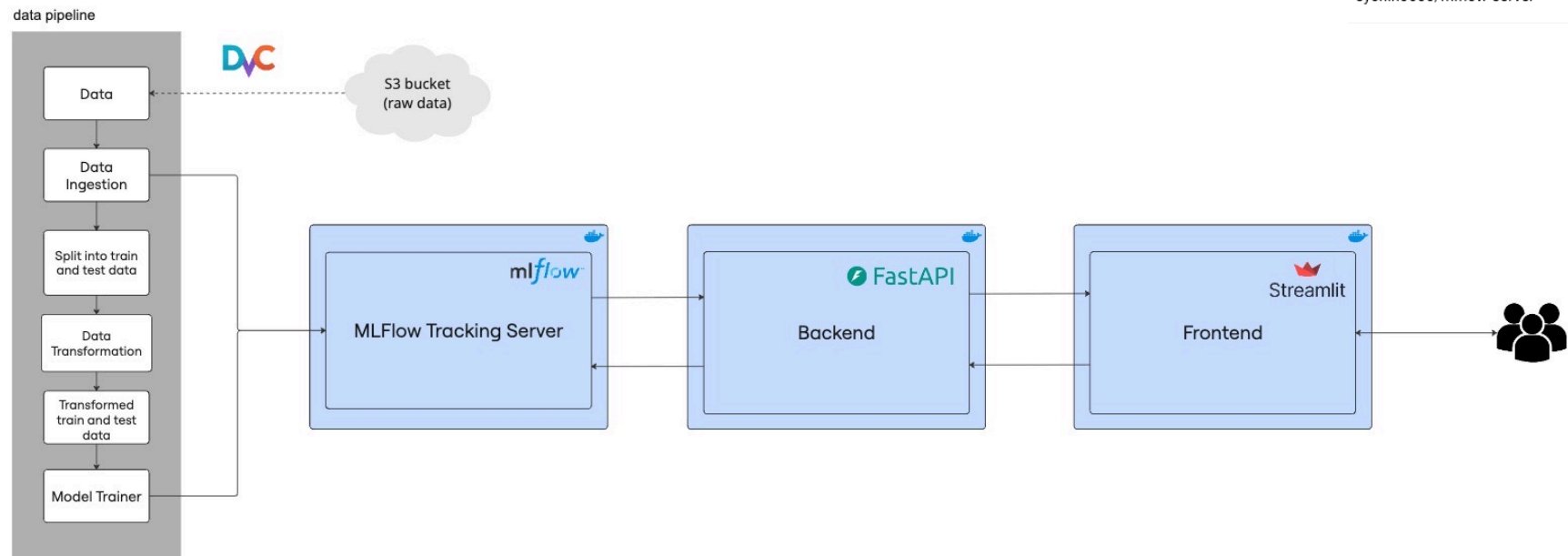
3. Pipeline Component — The Data & Model Pipeline

4. Infrastructure & Deployment— Scaling Across Environments



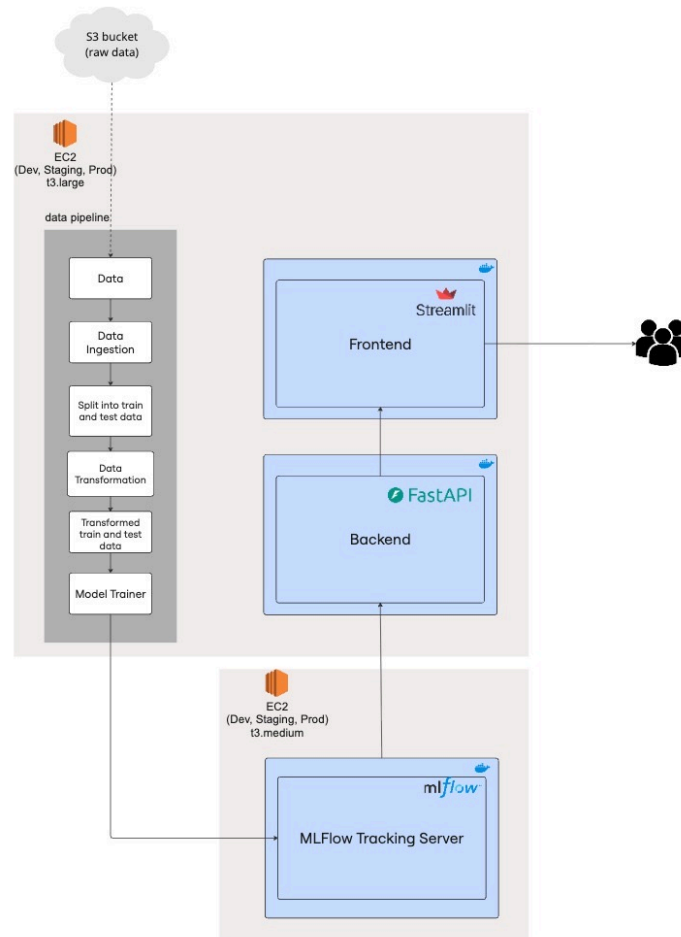
Demo

4.1 Docker containerisation

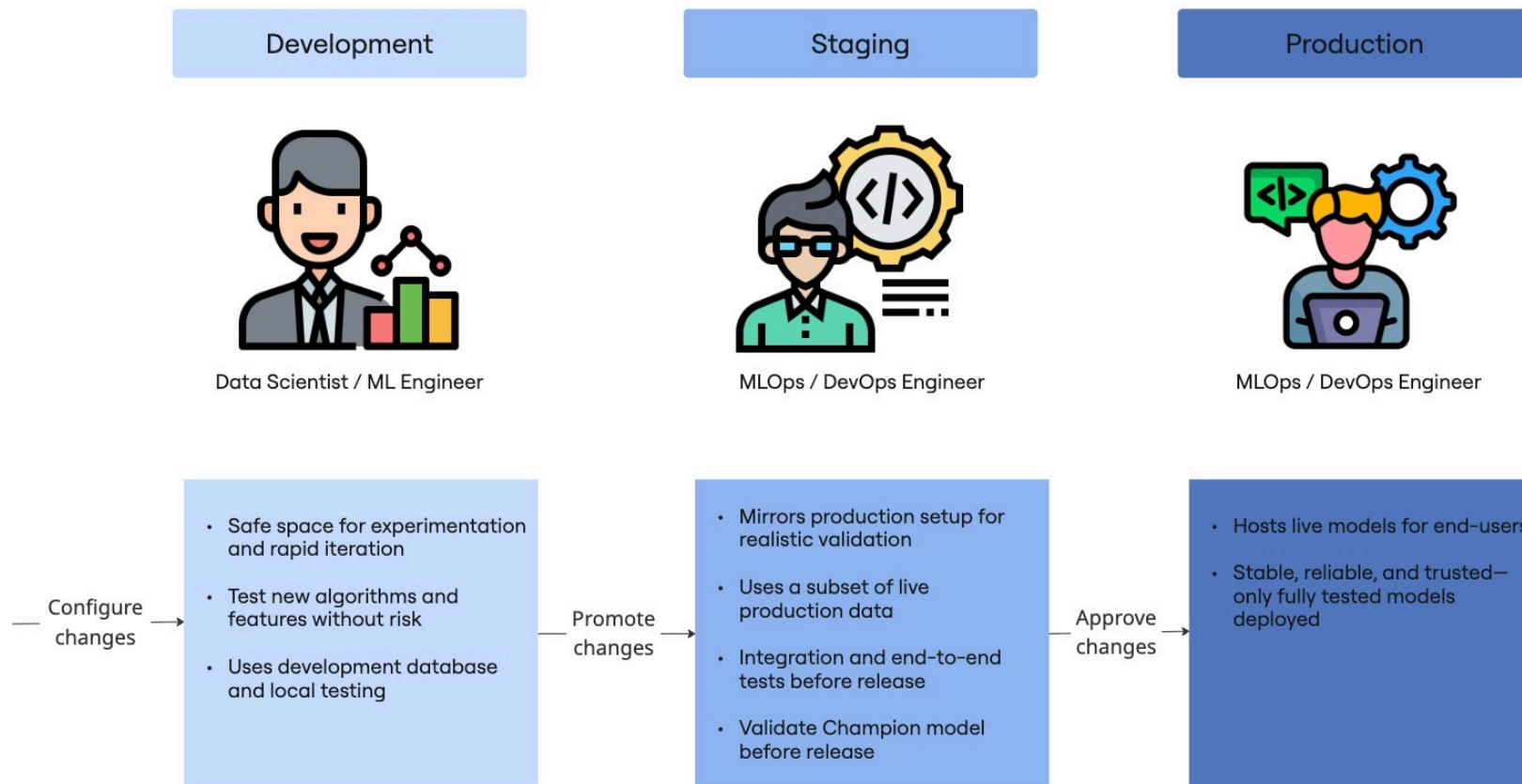


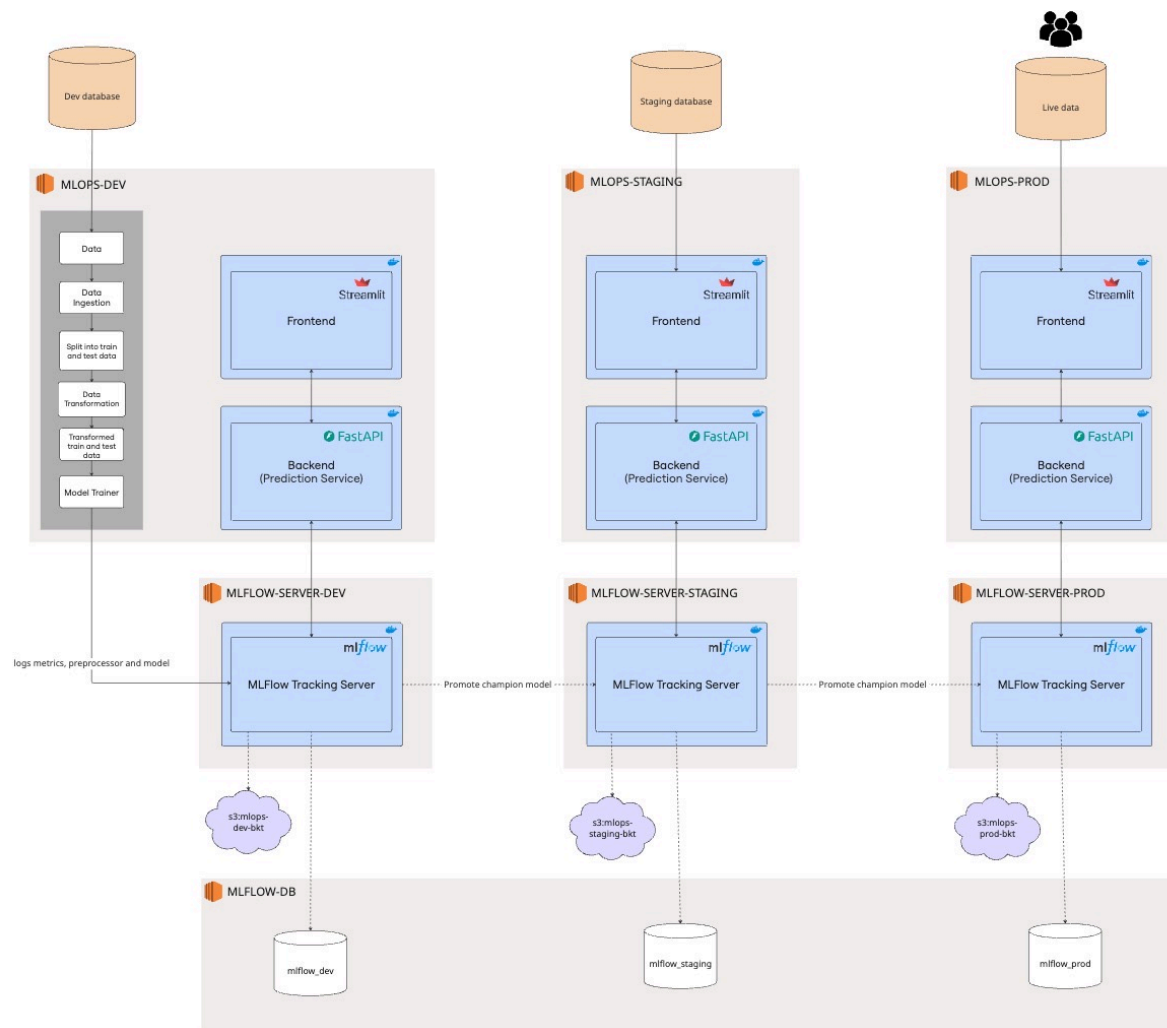
Name	hub
sychin0606/frontend	
sychin0606/backend	
sychin0606/mlflow-server	

4.2 Virtual Machines: AWS EC2



4.3 Multi-Environment: Dev, Staging, Production





Multi-Environment: Dev, Staging, Production

DSA4288 MLOPS FOR TRADITIONAL ML HONOURS PROJECT
PROJECT MACHINES

\$ MONTHLY

Environment	Name	Status	AMI	Instance Type	RAM	vCPUs	OS	Storage
MLOps-FYP	MLOPS-FYP-MACHINE	STOPPED	Canonical, Ubuntu 22.04	T3.LARGE	8 GiB	2	Ubuntu Server 22.04 LTS	35 GiB SSD
	MLOPS-FYP-MACHINE/DEV/SDA1						Ubuntu Server 22.04 LTS	40 GiB SSD
	MLOPS-FYP-MACHINE/DEV/SDD						Ubuntu Server 22.04 LTS	40 GiB SSD
MLFlow-DB	MLFLOW-DB	STOPPED	Canonical, Ubuntu 22.04	T3.MEDIUM	4 GiB	2	Ubuntu Server 22.04 LTS	10 GiB SSD
	MLFLOW-DB/DEV/SDA1						Ubuntu Server 22.04 LTS	10 GiB SSD
	MLFLOW-DB/DEV/SDD						Ubuntu Server 22.04 LTS	40 GiB SSD
MLOps-Dev	MLOPS-DEV	STOPPED	Canonical, Ubuntu 22.04	T3.MEDIUM	4 GiB	2	Ubuntu Server 22.04 LTS	25 GiB SSD
	MLOPS-DEV/DEV/SDA1						Ubuntu Server 22.04 LTS	25 GiB SSD
	MLOPS-DEV/DEV/SDD						Ubuntu Server 22.04 LTS	40 GiB SSD
MLFlow-Server-Dev	MLFLOW-SERVER-DEV	STOPPED	Canonical, Ubuntu 22.04	T3.MEDIUM	4 GiB	2	Ubuntu Server 22.04 LTS	10 GiB SSD
	MLFLOW-SERVER-DEV/DEV/SDA1						Ubuntu Server 22.04 LTS	10 GiB SSD
	MLFLOW-SERVER-DEV/DEV/SDD						Ubuntu Server 22.04 LTS	40 GiB SSD
MLOps-Staging	MLOPS-STAGING	STOPPED	Canonical, Ubuntu 22.04	T3.MEDIUM	4 GiB	2	Ubuntu Server 22.04 LTS	25 GiB SSD
	MLOPS-STAGING/DEV/SDA1						Ubuntu Server 22.04 LTS	25 GiB SSD
	MLOPS-STAGING/DEV/SDD						Ubuntu Server 22.04 LTS	40 GiB SSD
MLFlow-Server-Staging	MLFLOW-SERVER-STAGING	STOPPED	Canonical, Ubuntu 22.04	T3.MEDIUM	4 GiB	2	Ubuntu Server 22.04 LTS	25 GiB SSD
	MLFLOW-SERVER-STAGING/DEV/SDA1						Ubuntu Server 22.04 LTS	25 GiB SSD
	MLFLOW-SERVER-STAGING/DEV/SDD						Ubuntu Server 22.04 LTS	40 GiB SSD
MLOps-Prod	MLOPS-PROD	STOPPED	Canonical, Ubuntu 22.04	R5.XLARGE	32 GiB	4	Ubuntu Server 22.04 LTS	25 GiB SSD
	MLOPS-PROD/DEV/SDA1						Ubuntu Server 22.04 LTS	25 GiB SSD
	MLOPS-PROD/DEV/SDD						Ubuntu Server 22.04 LTS	40 GiB SSD
MLFlow-Server-Prod	MLFLOW-SERVER-PROD	STOPPED	Canonical, Ubuntu 22.04	T3.MEDIUM	4 GiB	2	Ubuntu Server 22.04 LTS	10 GiB SSD
	MLFLOW-SERVER-PROD/DEV/SDA1						Ubuntu Server 22.04 LTS	10 GiB SSD
	MLFLOW-SERVER-PROD/DEV/SDD						Ubuntu Server 22.04 LTS	40 GiB SSD

4.4 How we separate Dev, Staging, Production

Different configuration usings different:

- EC2 instances
- Backend databases
- S3 buckets
- .env files
- Docker compose files

Component / Purpose	Dev	Staging	Prod
EC2 Instance Name	MLFLOW-SERVER-DEV	MLFLOW-SERVER-STAGING	MLFLOW-SERVER-PROD
PostgreSQL Database	mlflow_dev	mlflow_staging	mlflow_prod
MLflow Artefacts S3	s3://mlops-dev-bkt/	s3://mlops-stag-bkt/	s3://mlops-prod-bkt/
Docker Compose File	docker-compose.mlflow-server-dev.yml	docker-compose.mlflow-server-staging.yml	docker-compose.mlflow-server-prod.yml
Environment File	.env.mlflow_dev	.env.mlflow_stag	.env.mlflow_prod

Table 4.2: MLflow Server Configuration Across Environments

Component / Purpose	Dev	Staging	Prod
EC2 Instance Name	MLOPS-DEV	MLOPS-STAGING	MLOPS-PROD
Raw Data S3 Bucket	s3://mlops-dev-raw/	s3://mlops-stag-raw/	s3://mlops-prod-raw/
Docker Compose File	docker-compose.dev.yml	docker-compose.staging.yml	docker-compose.prod.yml
Environment File	.env.dev_machine	.env.stag_machine	.env.prod_machine
Download data	make download-data-dev	make download-data-staging	make download-data-prod
Spin up services	make up-dev	make up-staging	make up-prod
Model promotion	make promote-model-dev	make promote-model-staging	-
Testing	make test-dev	make test-staging	make test-prod

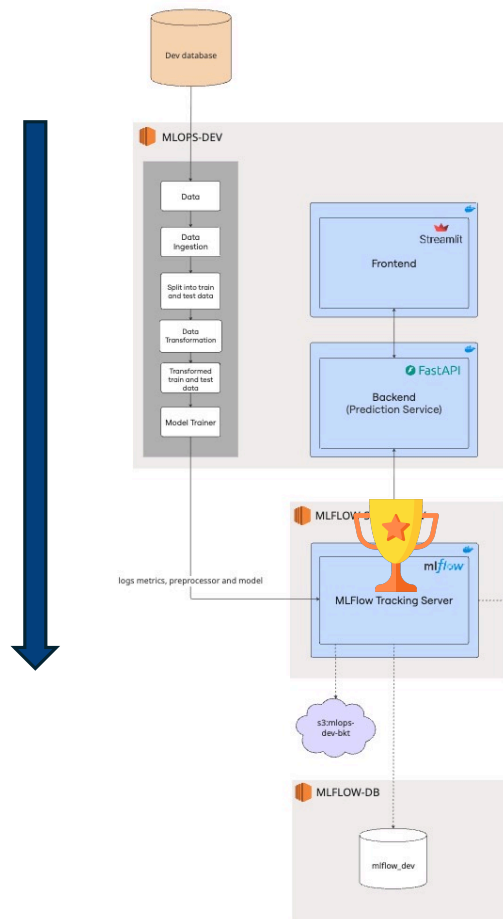
Table 4.3: Backend + Frontend Configuration Across Environments

How do our Champion Model move through the 3 stages to reach end users?

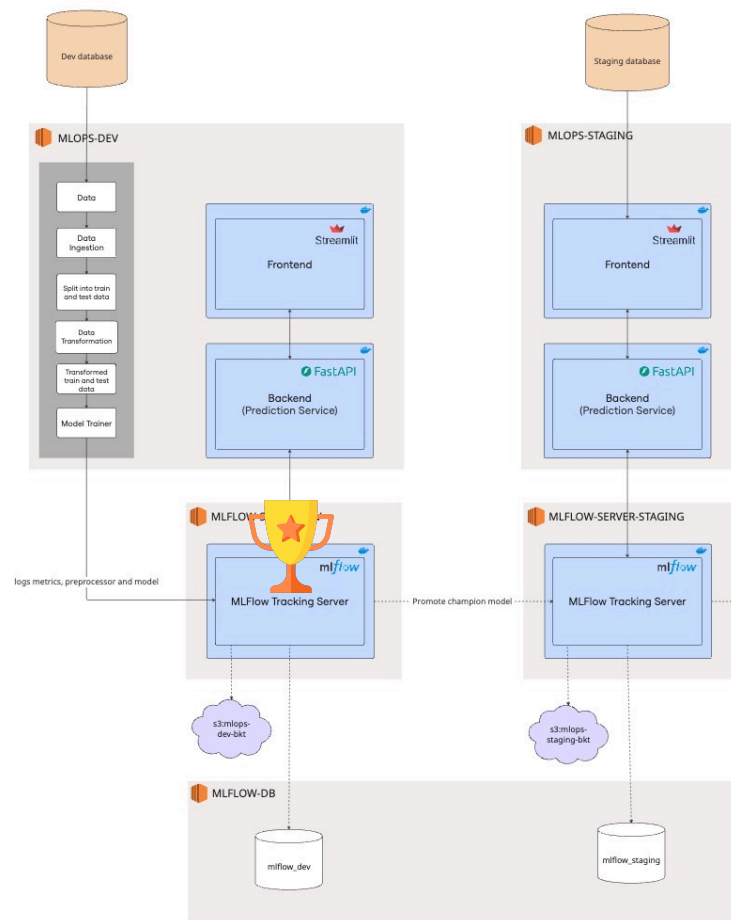


Model Promotion Workflow

4.5 Model Promotion Workflow Dev → Staging → Production

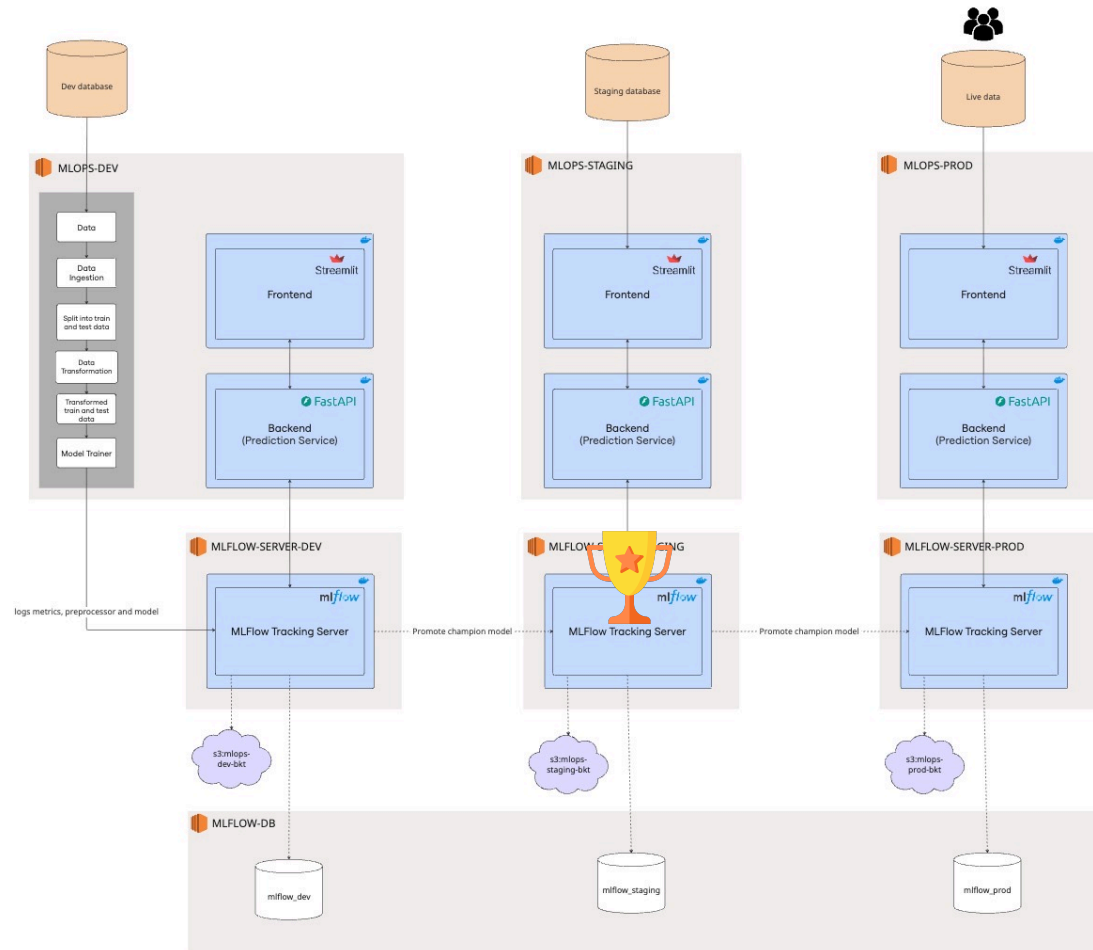


4.5 Model Promotion Workflow Dev → Staging → Production



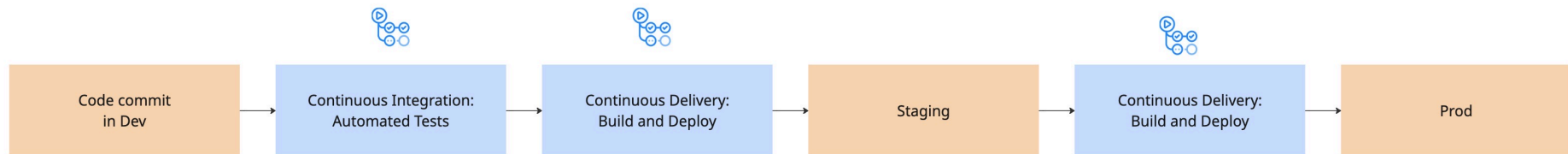
```
promote-model-dev:  
$(MAKE) download-champion-dev  
$(MAKE) upload-champion-to-staging  
AWS_PROFILE=dev-bkt AWS_DEFAULT_PROFILE=dev-bkt
```

4.5 Model Promotion Workflow Dev → Staging → Production



```
promote-model-staging:  
$(MAKE) download-champion-staging  
$(MAKE) upload-champion-to-prod  
AWS_PROFILE=stag-bkt AWS_DEFAULT_PROFILE=stag-bkt
```

How do we automate the journey from code and models to production—while catching errors before they reach users?



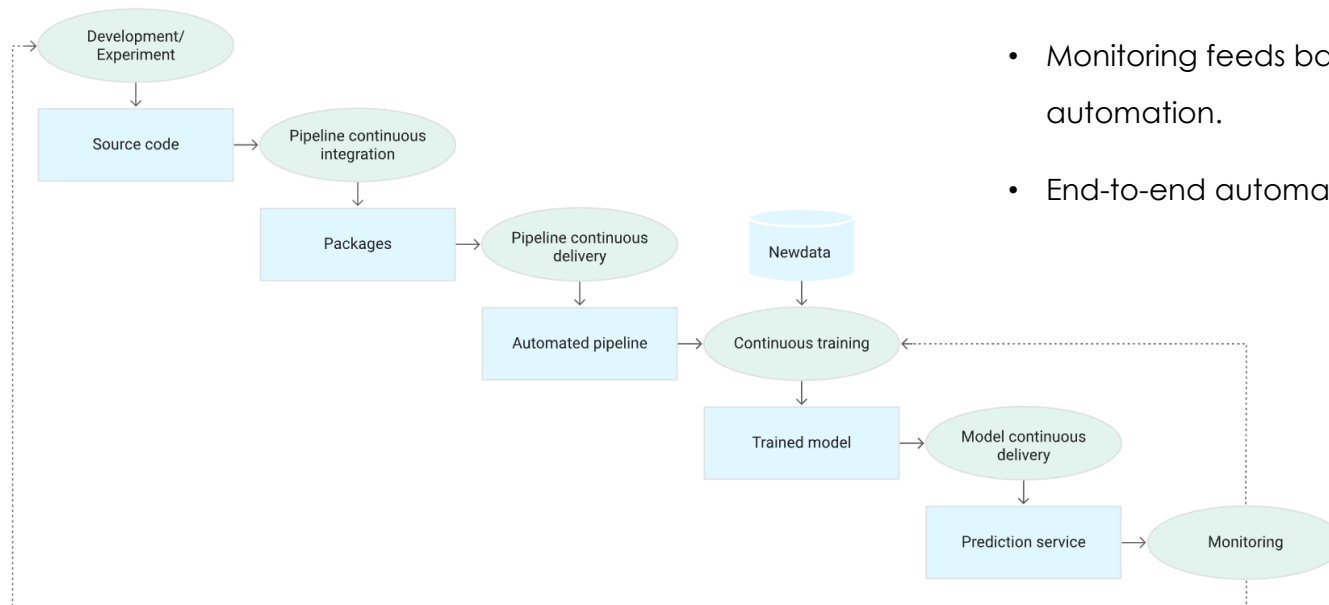
4.6 CI/CD

Automating Integration, Deployment, and Testing

- **CI (Continuous Integration):** Automatically tests and validates every code/model change.
- **CD (Continuous Deployment):** Packages and deploys applications/models to staging and production.
- Ensures reliability, reproducibility, and fast delivery across all environments.
- Reduces manual errors and mirrors industry best practices.

4.6.1 Gold Standard: Fully Automated ML CI/CD (Google Cloud)

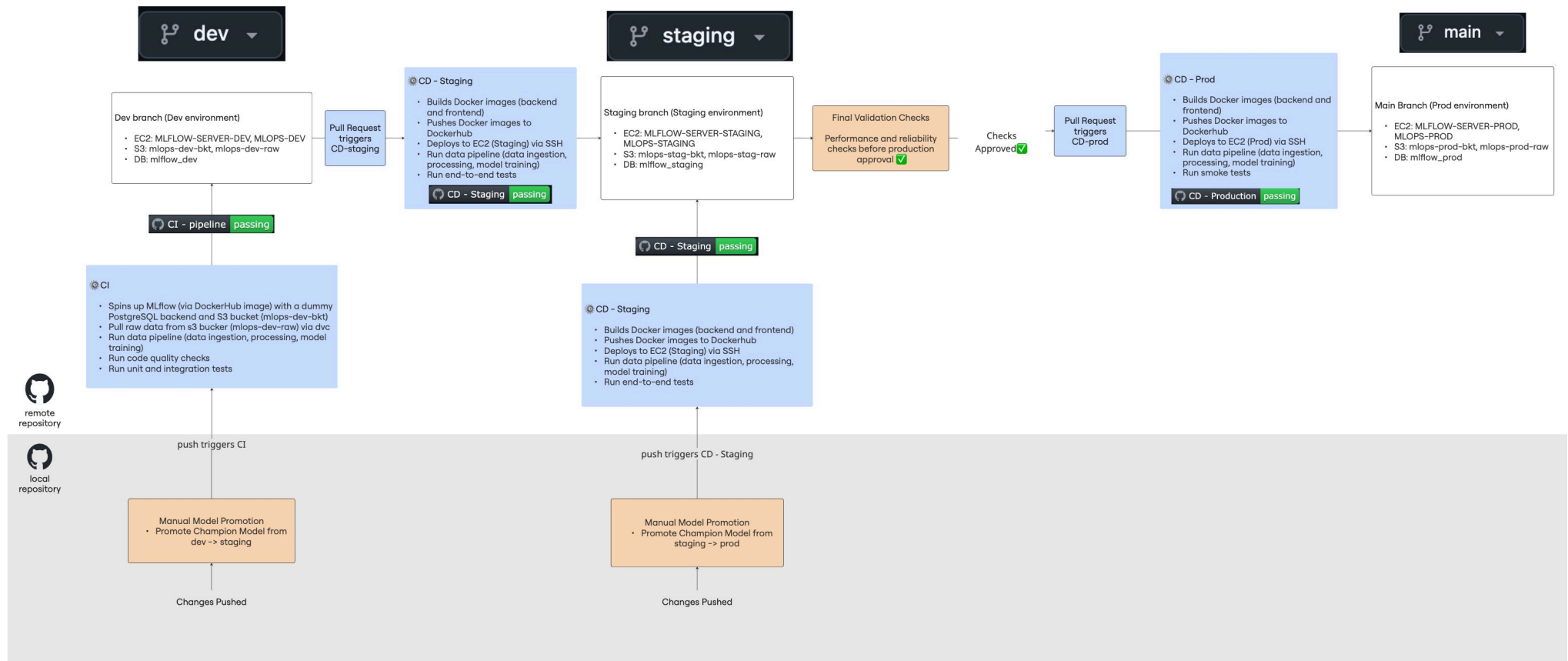
- Continuous integration and delivery for both code and models.
- Automated retraining triggered by new data.
- Monitoring feeds back to retraining pipeline for true ML automation.
- End-to-end automation: minimal manual intervention.



Source: Google cloud

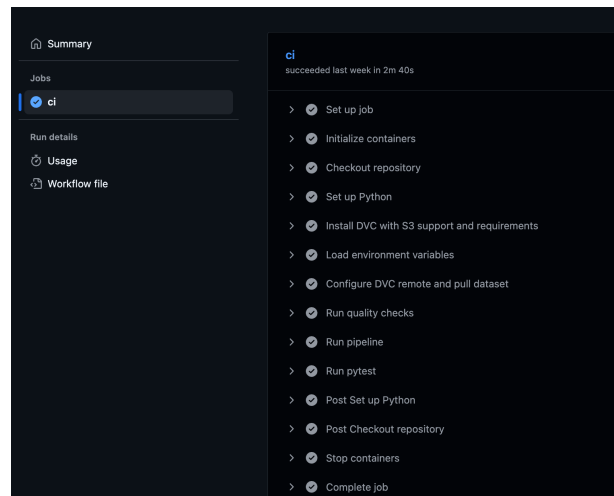
- Legend
- Blue Box - Automated (GitHub Actions)
 - Orange Box - Manual Step
 - White Box - Environment

4.6.2 My Implementation: Combining Manual and Automated steps



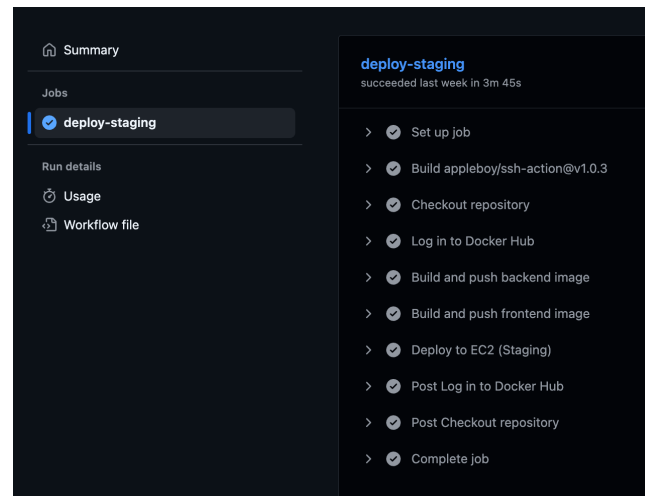
4.6.3 GitHub Actions View

CI



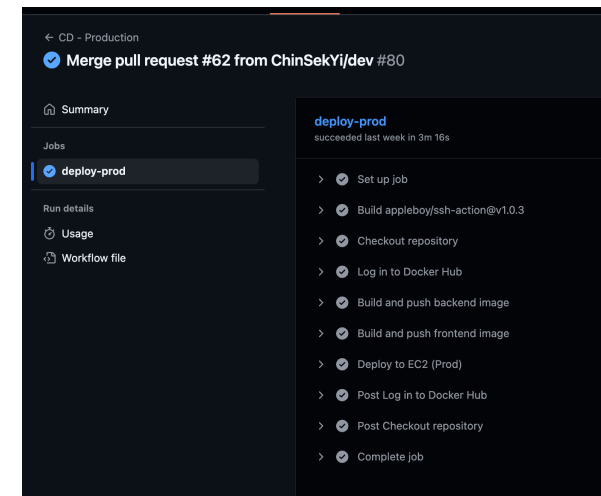
The screenshot shows the GitHub Actions interface for a CI workflow. The left sidebar has a 'Summary' section with links to 'Jobs', 'Run details', 'Usage', and 'Workflow file'. The 'Jobs' section is expanded, showing a single job named 'ci' with a status of 'succeeded last week in 2m 40s'. The main area displays a list of steps for the 'ci' job, each with a status icon and a name: 'Set up job', 'Initialize containers', 'Checkout repository', 'Set up Python', 'Install DVC with S3 support and requirements', 'Load environment variables', 'Configure DVC remote and pull dataset', 'Run quality checks', 'Run pipeline', 'Run pytest', 'Post Set up Python', 'Post Checkout repository', 'Stop containers', and 'Complete job'.

CD-Staging



The screenshot shows the GitHub Actions interface for a CD-Staging workflow. The left sidebar has a 'Summary' section with links to 'Jobs', 'Run details', 'Usage', and 'Workflow file'. The 'Jobs' section is expanded, showing a single job named 'deploy-staging' with a status of 'succeeded last week in 3m 45s'. The main area displays a list of steps for the 'deploy-staging' job, each with a status icon and a name: 'Set up job', 'Build appleboy/ssh-action@v1.0.3', 'Checkout repository', 'Log in to Docker Hub', 'Build and push backend image', 'Build and push frontend image', 'Deploy to EC2 (Staging)', 'Post Log in to Docker Hub', 'Post Checkout repository', and 'Complete job'.

CD-Prod



The screenshot shows the GitHub Actions interface for a CD-Prod workflow. The left sidebar has a 'Summary' section with links to 'Jobs', 'Run details', 'Usage', and 'Workflow file'. The 'Jobs' section is expanded, showing a single job named 'deploy-prod' with a status of 'succeeded last week in 3m 16s'. The main area displays a list of steps for the 'deploy-prod' job, each with a status icon and a name: 'Set up job', 'Build appleboy/ssh-action@v1.0.3', 'Checkout repository', 'Log in to Docker Hub', 'Build and push backend image', 'Build and push frontend image', 'Deploy to EC2 (Prod)', 'Post Log in to Docker Hub', 'Post Checkout repository', and 'Complete job'.

Roadmap

1. MLOps Motivation

2. System Overview & Use Case — Building the Foundation

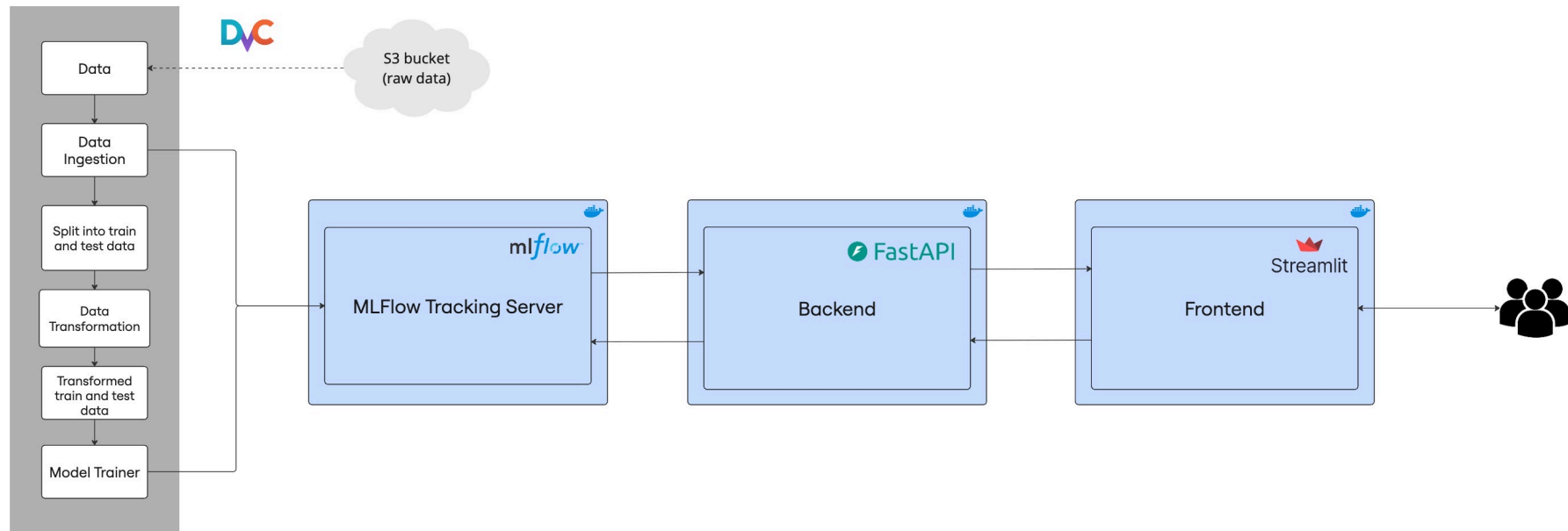
3. Pipeline Component — The Data & Model Pipeline

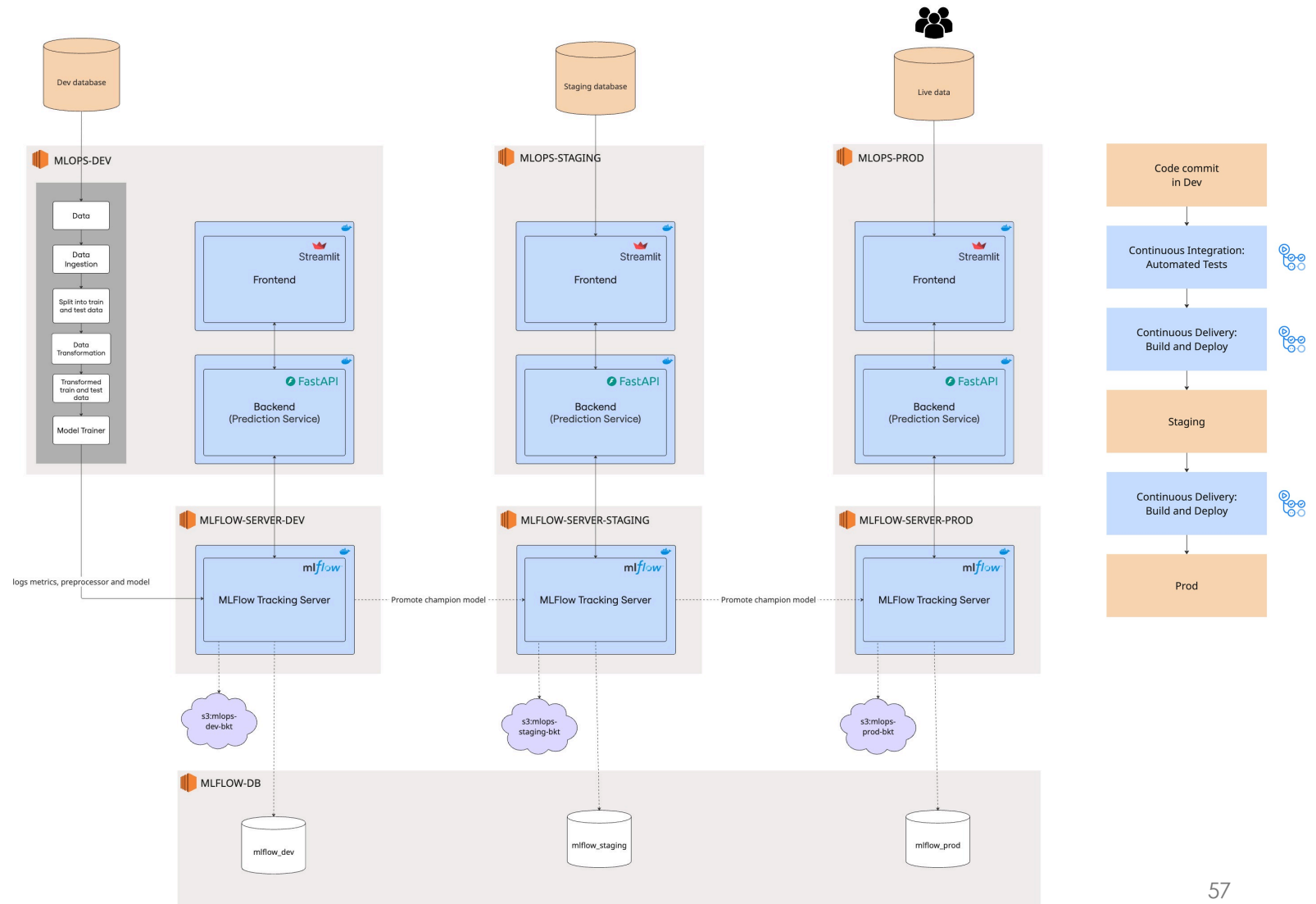
4. Infrastructure & Deployment— Scaling Across Environments



Demo

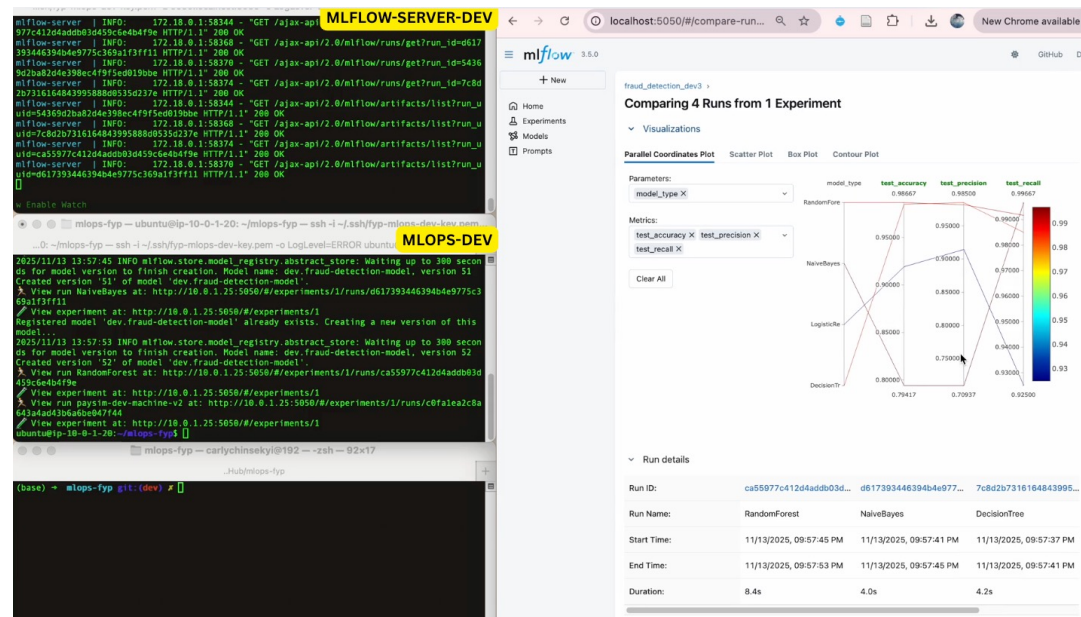
data pipeline





Demo 1:

Data Scientist's Development workflow



```
(base) → mlops-fyp git:(dev) ✕
```

I

```
mlops-fyp — carlychinsekyi@192 — zsh — 92x18
```

```
..Hub/mlops-fyp
```

```
(base) → mlops-fyp git:(dev) ✕
```

```
mlops-fyp — carlychinsekyi@192 — zsh — 92x17
```

```
..Hub/mlops-fyp
```

```
(base) → mlops-fyp git:(dev) ✕
```

google.com/webhp?hl=en&ictx... New Chrome available

About Store

Gmail Images

Google



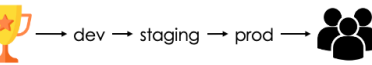
AI Mode

Google Search

I'm Feeling Lucky

Google offered in: 简体中文 Bahasa Melayu தமிழ்

Singapore



DEV ENV

(base) → ~ []

(base) → ~ []

carlychinsekyi — carlychinsekyi@192 — zsh — 104x29

mlops-fyp — carlychinsekyi@192 — zsh — 104x29

..Hub/mlops-fyp

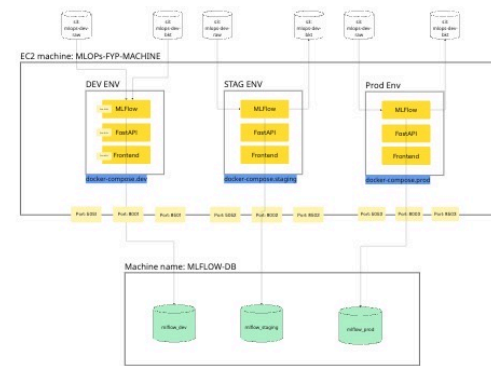
(base) → ~ []

(base) → mlops-fyp git:(dev) ✕ []

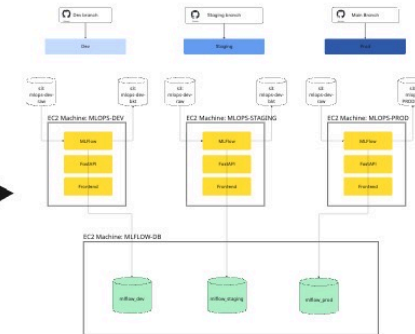
Evaluation

- **Development Velocity:**
 - Experiment setup time reduced from 5 hours to under 5 minutes (95% faster).
 - Model deployment became near-instant via MLflow registry.
- **Reproducibility & Collaboration:**
 - DVC + MLflow enabled full experiment reproducibility and seamless sharing.
 - Central MLflow server replaced error-prone manual tracking.
- **Error Reduction:**
 - Automated pipelines eliminated code/data/config integration issues.
 - Early bug detection via Docker builds and CI/CD validation.
- **Knowledge Transfer:**
 - Standardised pipeline lowered onboarding time for new team members.

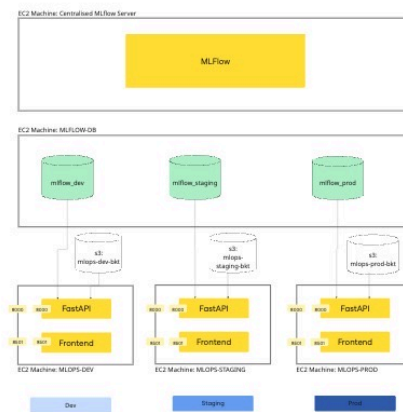
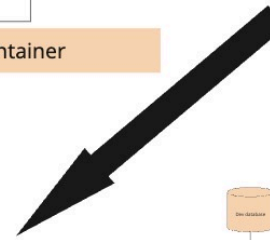
Project Evolution



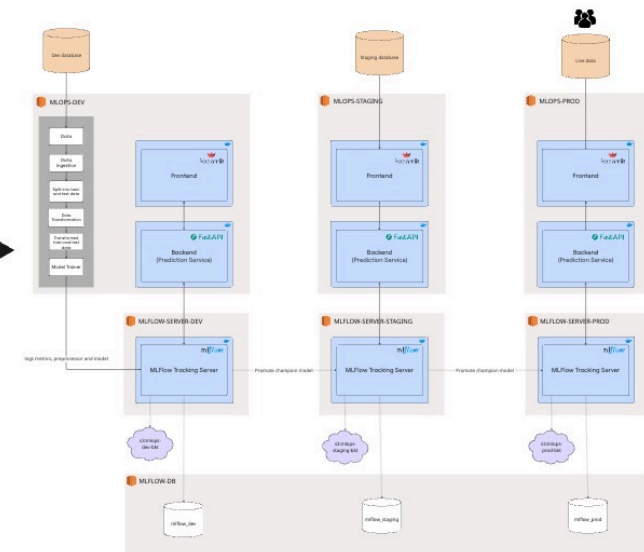
Plan A: Single EC2, Multi-Container



Plan B: Multi-EC2 Isolation



Plan C: Central MLflow Server



Plan D: Final Multi-Environment Setup

Challenges & Limitations

Challenges

- Setting up multi-environment AWS infrastructure.
- Docker and EC2 networking and connectivity between services required troubleshooting.
- MLflow configuration for artifact storage and experiment migration was tricky.
- CI/CD failures often requires many git commits to resolve.

Limitations

- No automated monitoring for data/model drift—manual checks only.
- CI/CD does not support automated retraining; retraining is manual.
- No feature store—features managed manually, limiting reusability.
- Several manual steps remain (retraining, model promotion); reducing these would improve reliability and scalability.

Future Work

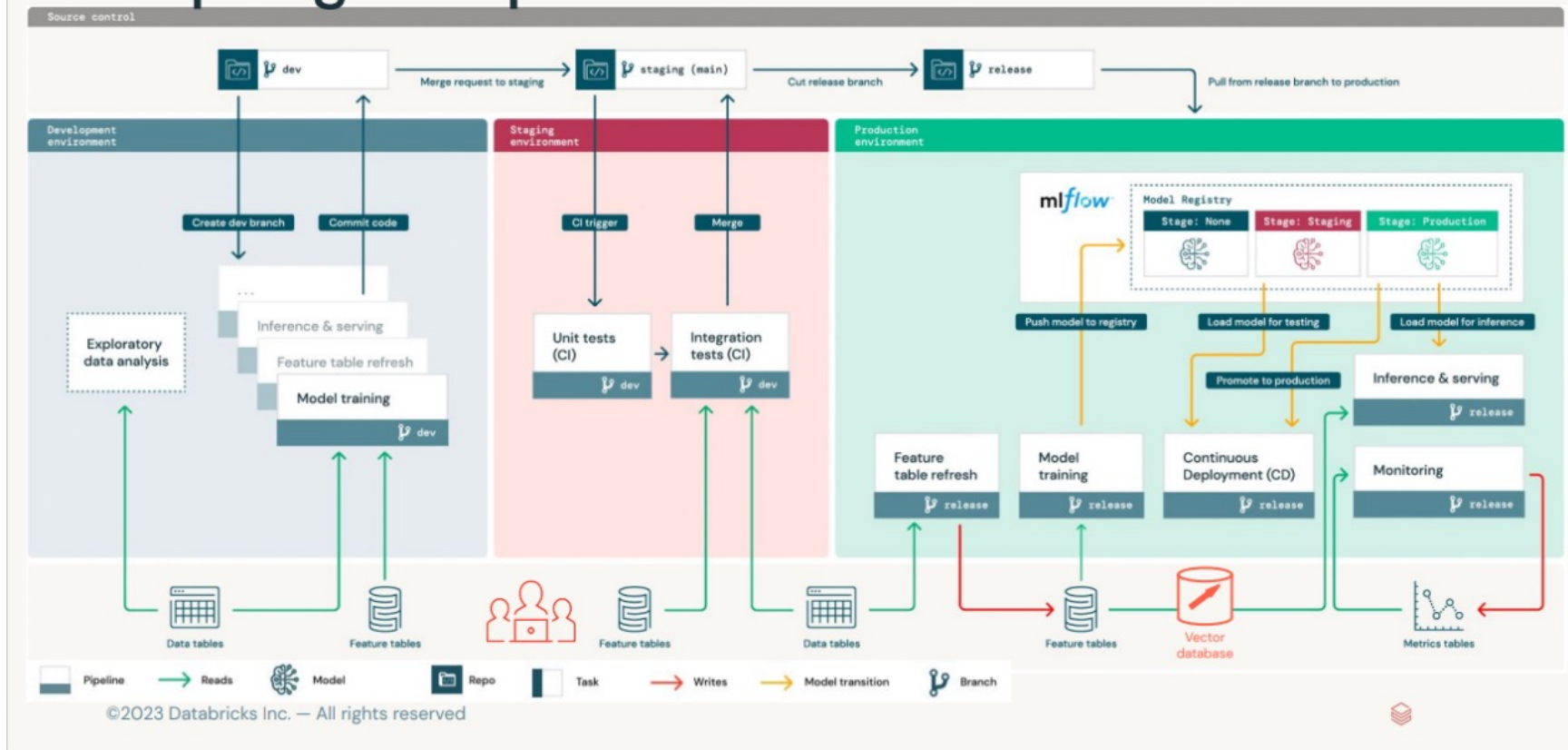
- Automated drift monitoring & alerting (Evidently AI, Grafana)
- CI/CD-triggered retraining & deployment (Jenkins)
- Centralized feature store for consistency & reuse
- Full automation of retraining & model promotion
- Auto-scaling for production workloads (Kubernetes)
- Support for deep learning frameworks (TensorFlow, PyTorch)
- Extend architecture for LLMOps & large language models

Conclusion

- Bridged the gap between academic ML and real-world impact
- Delivered a reproducible, production-ready MLOps platform
- Automated deployment, monitoring, and environment separation
- Enabled models to move from notebooks to serving real users
- Set the stage for future innovation (deep learning, LLMOps)

Adapting MLOps for LLMs

Some things change—but even more remain similar.



Thank You

Any Questions?

By: Chin Sek Yi

Project repository: <https://github.com/ChinSekYi/mlops-fyp>

Please try the Fraud Detection Model!

<http://3.1.190.42:8501/>

SCAN ME

